



Université de Tunis El Manar

Faculté des Sciences de Tunis

Département des Sciences de l'Informatique



ChargeTrackr

Rapport de stage d'été

Développement d'un outil de visualisation et de suivi de la disponibilité des bornes de recharge électrique

Réalisé par

Wajdi Ben Abdeljelil

Encadré en entreprise par

M. Bassem Soua

Formation

Cycle ingénieur en génie
logiciel (IGL3)

Organisme d'accueil



TAC-TIC Tunisie

Du 1^{er} juillet au 31 août 2026

Année universitaire 2025–2026

Remerciements

Je tiens à remercier l'entreprise TAC-TIC Tunisie pour son accueil et pour l'opportunité de réaliser ce stage dans un contexte professionnel lié à la mobilité électrique et aux systèmes logiciels distribués.

J'adresse mes remerciements à M. Bassem Soua, encadrant en entreprise, pour son accompagnement, ses orientations et les échanges consacrés à la compréhension des besoins du projet.

Je remercie également les enseignants du Département des Sciences de l'Informatique de la Faculté des Sciences de Tunis pour les connaissances acquises durant ma première année du cycle ingénieur en génie logiciel.

Périmètre prévu

Cette page sera relue et complétée avant la version finale afin d'y intégrer les personnes et organismes qui auront contribué à la réalisation et à la validation du stage.

Table des matières

Remerciements	i
Liste des acronymes	xii
Introduction générale	1
1 État de l’art et fondements métiers	2
1.1 Cadre du projet	2
1.1.1 Cadre académique	2
1.1.2 Présentation de l’organisme d’accueil	3
1.1.3 Présentation du domaine d’application	3
1.1.3.1 Mobilité électrique et infrastructures de recharge	3
1.1.3.2 Supervision d’un réseau de bornes	4
1.1.3.3 OCPP et interopérabilité	4
1.1.3.4 Disponibilité et qualité de service	5
1.2 Problématique	6
1.3 Étude de l’existant	7
1.3.1 Solutions existantes	7
1.3.1.1 ChargePoint	7
1.3.1.2 Driivz	8
1.3.1.3 Monta	8
1.3.1.4 CitrineOS	8
1.3.2 Standards, outils et simulateurs	8
1.3.2.1 Référentiel OCPP et sécurité	8
1.3.2.2 Simulation des stations	9
1.3.2.3 Cartographie et paiement	9

1.3.3	Synthèse comparative	9
1.4	Solution proposée	10
1.4.1	Périmètre fonctionnel	10
1.4.2	Espaces authentifiés par rôle	11
1.4.3	Chaîne verticale de fonctionnement	11
1.4.4	Périmètre du MVP et extensions	12
1.5	Méthodologie de travail	12
1.5.1	Approche itérative et incrémentale	12
1.5.2	Cycle de traitement d'un bloc	13
1.5.3	Traçabilité et contrôle de la qualité	14
2	Spécification et analyse des besoins	15
2.1	Spécification des besoins	15
2.1.1	Identification des acteurs	15
2.1.2	Besoins fonctionnels	17
2.1.2.1	Fonctions communes aux utilisateurs authentifiés	17
2.1.2.2	Super Administrateur	18
2.1.2.3	Administrateur d'organisation	18
2.1.2.4	Opérateur	19
2.1.2.5	Technicien	20
2.1.2.6	Client	20
2.1.2.7	Visiteur et traitements automatiques	21
2.1.3	Besoins non fonctionnels	21
2.2	Planification du projet	24
2.2.1	Méthode de priorisation et d'estimation	24
2.2.2	Product Backlog révisé	24
2.2.3	Découpage en incréments	27
2.2.4	Matrice de traçabilité fonctionnelle	29
2.3	Analyse globale des cas d'utilisation	30

2.3.1	Diagramme global	30
2.3.2	Catalogue des cas majeurs	32
2.3.3	UC-01 – Demander une démonstration et provisionner une organisation	33
2.3.4	UC-02 – Inviter et activer un employé	34
2.3.5	UC-03 – Mettre en service une station OCPP	35
2.3.6	UC-04 – Calculer la disponibilité	36
2.3.7	UC-05 – Effectuer une recharge et payer	37
2.3.8	UC-06 – Traiter une alerte et une intervention	39
2.3.9	UC-07 – Gérer le cycle commercial d’une organisation	40
2.3.10	UC-08 – Générer et échanger un rapport	41
3	Conception	43
3.1	Conception générale	43
3.1.1	Style architectural retenu	43
3.1.2	Architecture technique globale	44
3.1.2.1	Application web React	46
3.1.2.2	API et cœur métier Laravel	46
3.1.2.3	Persistance, asynchronisme et temps réel	46
3.1.2.4	Passerelle OCPP	47
3.1.2.5	Services externes et adaptateurs	48
3.1.3	Architecture de déploiement	48
3.1.4	Architecture logicielle	50
3.1.4.1	Organisation du frontend	51
3.1.4.2	Organisation du backend	51
3.2	Conception détaillée	52
3.2.1	Identité, demandes de démonstration et invitations	52
3.2.2	Mise en service d’une station	53
3.2.3	Ingestion OCPP et disponibilité calculée	54

3.2.3.1	Séparation entre preuve brute et projection métier . . .	54
3.2.3.2	Règles de disponibilité	54
3.2.4	Parcours de recharge et transaction OCPP	55
3.2.4.1	Assistant côté client	55
3.2.4.2	Tentative, session et transaction	56
3.2.5	Paieement et reçu	58
3.2.6	Alertes, interventions et maintenance	58
3.2.7	Tarification, plans et gestion commerciale	59
3.2.8	Rapports, documents et exports	60
3.3	Modèle de données et diagrammes de classes	60
3.3.1	Principes de modélisation	60
3.3.2	Identité, actifs et recharge	61
3.3.2.1	Organisation et utilisateur	63
3.3.2.2	Station et connecteur	63
3.3.2.3	Transaction et session	63
3.3.3	Exploitation et commerce	63
3.3.3.1	Agrégat d'exploitation	65
3.3.3.2	Agrégat de tarification client	65
3.3.3.3	Agrégat commercial SaaS	65
3.3.4	Contraintes et invariants principaux	65
3.4	Sécurité et isolation multi-tenant	66
3.4.1	Authentification de l'application web	66
3.4.2	Autorisation et périmètre organisationnel	67
3.4.3	Sécurité des frontières OCPP	68
3.4.4	Sécurité du paiement	68
3.4.5	Validation, limitation et gestion des erreurs	69
3.4.6	Secrets, fichiers et journalisation	69
3.4.7	Analyse synthétique des risques	70
3.4.8	Limites de sécurité du prototype	71

4	Réalisation et validation	72
4.1	Environnement et organisation de la réalisation	72
4.1.1	Poste, outils et versions	72
4.1.2	Structure du dépôt	74
4.1.3	Processus de développement	74
4.1.4	Gestion incrémentale avec Git	75
4.1.5	État synthétique des incréments	76
4.2	Réalisation des fondations techniques	77
4.2.1	INC-01 : entrée, identité et multi-tenant	77
4.2.1.1	Parcours d'entrée	77
4.2.1.2	Isolation organisationnelle	77
4.2.1.3	Onboarding orienté vers la première valeur	78
4.2.2	INC-02 : stations, connecteurs et cartographie	79
4.2.2.1	Référentiel et vues adaptées	79
4.2.2.2	Commissioning atomique	80
4.2.3	INC-03 : OCPP et disponibilité calculée	82
4.2.3.1	Connexion de la borne simulée	82
4.2.3.2	Règles de projection	83
4.2.3.3	Commandes et télémétrie	84
4.3	Réalisation des parcours métier	85
4.3.1	INC-04 : exploitation terrain	85
4.3.1.1	Alertes et qualification	85
4.3.1.2	Interventions, maintenance et SLA	86
4.3.2	INC-05 : recharge, transaction et paiement	86
4.3.2.1	Assistant de recharge client	86
4.3.2.2	Cycle de session	87
4.3.2.3	Adaptateur de paiement simulé	87
4.3.3	INC-06 : tarification et commerce	88
4.3.3.1	Tarifs et plans de recharge	88

4.3.3.2	Abonnement SaaS de l'organisation	88
4.3.4	INC-07 : pilotage, reporting et gouvernance	89
4.3.4.1	Tableaux de bord par rôle	89
4.3.4.2	Rapports et échanges internes	90
4.3.4.3	Administration globale	91
4.3.5	INC-08 : exports et évolutions différées	92
4.4	Intégration et validation	92
4.4.1	Intégration bout en bout	92
4.4.2	Stratégie de tests	93
4.4.3	Résultats mesurés	93
4.4.4	Anomalies représentatives et non-régression	94
4.4.5	Performance et robustesse	95
4.4.6	Sécurité appliquée	95
4.4.7	Limites de la validation	96
4.4.8	Traçabilité avec le backlog	96
4.5	Documentation technique produite et restante	97
5	Captures des interfaces	99
5.1	Accueil, authentification et onboarding	99
5.2	Tableaux de bord et supervision	100
5.3	Bornes, connecteurs et cartographie	101
5.4	Recharge, sessions et paiement	102
5.5	Alertes, interventions et maintenance	103
5.6	Administration, gestion commerciale et rapports	104
5.7	Conclusion	105
	Conclusion générale	106

Table des figures

2.1	Diagramme global des cas d'utilisation de ChargeTrackr	31
3.1	Architecture technique globale de ChargeTrackr	45
3.2	Architecture de déploiement de l'environnement local	49
3.3	Provisionnement d'une organisation à partir d'une demande de démonstration	52
3.4	Chaîne de traitement d'un événement OCPP	54
3.5	Séquence de recharge distante et paiement simulé	57
3.6	Cycle opérationnel des alertes, interventions et maintenances	59
3.7	Diagramme de classes conceptuel du noyau station-recharge	62
3.8	Diagramme de classes conceptuel des domaines d'exploitation et commerciaux	64
4.1	Dernière étape de l'onboarding de l'administrateur	79
4.2	Inventaire des stations avec indicateurs de disponibilité	80
4.3	Première étape du commissioning guidé d'une station	81
4.4	Fiche d'une station reliée au simulateur SAP	82
4.5	Télémétrie vérifiée sans génération de mesures fictives	84
4.6	Centre opérationnel des alertes	85
4.7	Parcours vertical d'une recharge	87
4.8	Suivi de l'abonnement commercial d'une organisation	89
4.9	Tableau de bord réel de l'administrateur	90
4.10	Studio de performance de l'administrateur	91
4.11	Composants traversés par le MVP vertical	92
5.1	Landing page ChargeTrackr	99
5.2	Connexion et création de compte	99

5.3	Onboarding personnalisé	100
5.4	Tableau de bord Super Administrateur	100
5.5	Tableau de bord Administrateur	100
5.6	Tableaux de bord Opérateur, Technicien et Client	101
5.7	Inventaire des bornes	101
5.8	Mise en service d'une borne	101
5.9	Détail et télémétrie d'une borne	102
5.10	Recherche d'une borne	102
5.11	Workflow de recharge	102
5.12	Session active et paiement	103
5.13	Gestion des alertes	103
5.14	Intervention technique	103
5.15	Maintenance	104
5.16	Gestion des utilisateurs et organisations	104
5.17	Tarifs et abonnements	104
5.18	Rapports et analyses	105
5.19	Profil et paramètres	105

Liste des tableaux

1.1	Règles métier de projection de la disponibilité	6
1.2	Comparaison synthétique des solutions étudiées	10
1.3	Organisation des incréments de réalisation	13
2.1	Acteurs humains de la plateforme	16
2.2	Acteurs secondaires et systèmes externes	17
2.10	Exigences non fonctionnelles	22
2.11	Product Backlog de ChargeTrackr	24
2.12	Répartition du backlog par priorité	27
2.13	Backlogs des incréments	28
2.14	Traçabilité entre backlog, API et interfaces	29
2.15	Catalogue des cas d'utilisation majeurs	32
3.1	Intégrations techniques et responsabilités	48
3.2	Processus nécessaires dans l'environnement de développement	50
3.3	Cibles de mise en service OCPP	53
3.4	Matrice de décision de disponibilité	54
3.5	Invariants structurants du modèle	65
3.6	Périmètre d'accès des rôles	67
3.7	Principales limites applicatives	69
3.8	Risques principaux et mesures de réduction	70
4.1	Environnement technique effectivement utilisé	73
4.2	Commandes principales de l'environnement local	75
4.3	État de réalisation au 28 juillet 2026	76
4.4	Exemples de règles de disponibilité réalisées	83

4.5	Niveaux de validation appliqués	93
4.6	Résultats de validation du dépôt	94
4.7	Preuves de réalisation par incrément	96

Liste des acronymes

Acronyme	Signification
API	Application Programming Interface
CSMS	Charging Station Management System
EV	Electric Vehicle
EVSE	Electric Vehicle Supply Equipment
OCPP	Open Charge Point Protocol
OAuth 2.0	Open Authorization 2.0
REST	Representational State Transfer
RFID	Radio Frequency Identification
SLA	Service Level Agreement
UI/UX	User Interface / User Experience
UML	Unified Modeling Language

Introduction générale

La transition vers la mobilité électrique entraîne le déploiement d'un nombre croissant de bornes de recharge. L'exploitation de ces équipements ne repose pas uniquement sur leur installation physique : elle exige également une supervision continue, une connaissance fiable de leur disponibilité et une coordination efficace entre les utilisateurs, les opérateurs et les techniciens.

Dans ce contexte, le présent stage d'été, réalisé au sein de TAC-TIC Tunisie, porte sur la conception et le développement de ChargeTrackr, une plateforme web consacrée à la visualisation et au suivi de la disponibilité des bornes de recharge électrique. Le projet associe gestion multi-organisation, échanges OCPP, cartographie, sessions de recharge, paiement simulé, alertes, interventions et reporting.

Le rapport conserve l'organisation en cinq chapitres du document de référence. Le premier chapitre introduit le cadre du projet, le domaine métier, la problématique, l'existant et la démarche retenue. Le deuxième formalise les acteurs, les besoins et la planification. Le troisième présente l'architecture et la conception détaillée. Le quatrième décrit la réalisation incrémentale, l'intégration et la validation. Enfin, le cinquième illustre les principales interfaces et les parcours adaptés aux différents rôles.

Périmètre prévu

La version rédigée précisera les objectifs pédagogiques et professionnels du stage, le périmètre validé avec l'encadrant, les contributions réalisées et les limites du travail présenté.

État de l’art et fondements métiers

Introduction

Ce chapitre situe le projet dans son cadre académique et professionnel, puis présente les fondements métiers indispensables à la compréhension d’une plateforme de gestion de bornes de recharge. Il précise notamment le rôle d’un système central de gestion, les échanges OCPP et la différence entre un état simplement enregistré et une disponibilité réellement déduite des événements du terrain. Une étude de solutions existantes permet ensuite de dégager les fonctions attendues, leurs limites au regard du stage et le positionnement retenu pour ChargeTrackr. Le chapitre se termine par la démarche de réalisation adoptée.

1.1 Cadre du projet

1.1.1 Cadre académique

Le présent travail est réalisé dans le cadre d’un stage d’été effectué du 1^{er} juillet au 31 août 2026 au sein de TAC-TIC Tunisie. Il intervient après la première année du cycle ingénieur en génie logiciel (IGL3), au Département des Sciences de l’Informatique de la Faculté des Sciences de Tunis, rattachée à l’Université de Tunis El Manar.

Le stage vise à mobiliser, dans un contexte professionnel, les compétences acquises en analyse des besoins, modélisation, développement d’applications web, conception d’API, gestion de bases de données, sécurité et tests logiciels. Au-delà de la réalisation d’interfaces, le sujet exige de traiter une problématique distribuée : l’état observé dans l’application dépend de messages émis par des équipements distants, de délais de communication et de règles métier vérifiables.

Le travail est encadré en entreprise par M. Bassem Soua. Le livrable principal est un prototype fonctionnel couvrant un parcours vertical : création et supervision d’une

station, réception de ses événements OCPP, calcul de sa disponibilité, conduite d'une session de recharge, paiement simulé et exploitation des informations produites.

1.1.2 Présentation de l'organisme d'accueil

TAC-TIC Tunisie est une société d'ingénierie indépendante fondée en 2012 et implantée à la Cité El Ghazala, dans le gouvernorat de l'Ariana. Sa présentation institutionnelle la positionne comme une entreprise spécialisée dans les technologies Open Source, les services réseau, le conseil, les solutions outillées et la formation. Elle mentionne également une expertise dans les services IP/MPLS destinés au secteur tunisien des télécommunications, ainsi que des activités de développement web et logiciel [1].

Ce positionnement rend pertinent un sujet associant systèmes communicants, interopérabilité et développement d'une plateforme web. Le projet ChargeTrackr s'inscrit dans cette logique : il ne se limite pas à afficher une liste d'équipements, mais doit établir une chaîne cohérente entre les bornes, les services backend et les différents espaces utilisateurs.

1.1.3 Présentation du domaine d'application

1.1.3.1 Mobilité électrique et infrastructures de recharge

Le développement de la mobilité électrique augmente la dépendance des conducteurs, des entreprises et des collectivités à des infrastructures de recharge accessibles et fiables. Selon l'Agence internationale de l'énergie, près de 1,8 million de points de recharge publics ont été ajoutés dans le monde en 2025, portant le parc total à plus de sept millions à la fin de cette année [2]. Dans ce contexte, la disponibilité opérationnelle devient un indicateur essentiel : une station référencée sur une carte mais hors ligne, en défaut ou entièrement occupée ne répond pas au besoin immédiat du conducteur.

Une infrastructure de recharge peut être décrite à plusieurs niveaux :

- la **station** ou borne représente l'équipement physique raccordé au réseau électrique et au système d'information ;
- le **connecteur** désigne un point de raccordement de la station. Une même borne peut proposer plusieurs connecteurs, de types et puissances différents ;
- la **session de recharge** enregistre l'utilisation d'un connecteur par un client entre un début et une fin, avec des mesures d'énergie et un coût ;

- l'**exploitant** supervise les équipements et leurs événements, tandis que le technicien traite les incidents et opérations de maintenance ;
- le **client** localise une borne, choisit un connecteur, lance sa recharge et consulte ses paiements ou abonnements.

Ces objets ne sont pas indépendants. L'état d'une station résulte de l'état de ses connecteurs et de sa capacité à communiquer. De même, une session client produit des informations utiles à la facturation, aux statistiques, à la maintenance et à l'évaluation de la qualité de service.

1.1.3.2 Supervision d'un réseau de bornes

Le système central chargé de piloter un réseau de recharge est couramment appelé *Charging Station Management System* (CSMS). Il reçoit les messages des bornes, maintient une représentation de leur état, conserve les transactions, autorise les utilisateurs et émet des commandes distantes. Il fournit ensuite aux acteurs humains des vues adaptées : carte publique, tableau de bord opérationnel, alertes, historique, interventions, tarifs, paiements et rapports.

La supervision doit distinguer au moins trois temporalités :

1. le **temps protocolaire**, constitué des connexions WebSocket et des messages OCPP reçus en continu ;
2. le **temps opérationnel**, durant lequel une anomalie est détectée, qualifiée, assignée puis résolue ;
3. le **temps analytique**, utilisé pour agréger l'énergie, la disponibilité, les revenus et les performances sur une période.

Une plateforme fiable ne doit donc pas utiliser le même mécanisme pour ces trois besoins. Les événements de terrain alimentent une projection courante ; les transitions importantes sont historisées ; les indicateurs sont calculés à partir des données vérifiées et d'une période explicitement choisie.

1.1.3.3 OCPP et interopérabilité

L'*Open Charge Point Protocol* (OCPP), maintenu par l'Open Charge Alliance, est un protocole ouvert qui standardise les échanges entre une station de recharge et un CSMS. Cette normalisation évite de lier la plateforme à un seul constructeur et permet de centraliser le contrôle d'équipements hétérogènes [3, 4].

Le projet adopte OCPP 1.6 JSON sur WebSocket. Une station s'identifie lors de sa connexion, puis échange des messages structurés avec la passerelle OCPP. Les messages principaux utilisés par ChargeTrackr sont :

- **BootNotification**, pour enregistrer le démarrage et négocier l'intervalle de heart-beat ;
- **Heartbeat**, pour apporter une preuve régulière de connectivité ;
- **StatusNotification**, pour transmettre l'état d'une station ou d'un connecteur et, le cas échéant, un code d'erreur ;
- **Authorize**, **StartTransaction**, **MeterValues** et **StopTransaction**, pour couvrir l'autorisation, le début, les mesures et la fin d'une transaction ;
- **RemoteStartTransaction**, **RemoteStopTransaction**, **Reset**, **UnlockConnector** et **ChangeAvailability**, pour les commandes distantes nécessaires aux parcours client et opérateur.

Les scénarios de conformité OCPP montrent qu'une recharge ne correspond pas à un simple changement booléen. Dans un scénario où le câble est branché avant l'autorisation, le connecteur passe notamment par l'état **Preparing**, puis une transaction est démarrée avant le passage à **Charging**. Une fin distante entraîne l'arrêt de transaction, l'état **Finishing**, puis le retour à **Available** [5].

Le protocole contribue à l'interopérabilité, mais ne définit pas à lui seul toutes les règles fonctionnelles de l'application. Le délai après lequel une borne est considérée hors ligne, la priorité d'une maintenance planifiée ou la manière d'agréger plusieurs connecteurs restent des décisions du système central.

1.1.3.4 Disponibilité et qualité de service

Dans ChargeTrackr, la disponibilité n'est pas uniquement une valeur saisie dans un formulaire. Elle constitue une **projection métier** recalculée à partir des preuves les plus récentes. L'Open Charge Alliance recommande justement d'exploiter la connexion WebSocket, le heartbeat et les messages **StatusNotification** pour améliorer la mesure du temps de fonctionnement [6].

La projection implémentée applique l'ordre de priorité suivant :

1. une désactivation manuelle rend la station **unavailable** ;
2. une maintenance planifiée la place en **maintenance** ;

3. une fermeture explicite de la connexion ou 90 secondes sans aucun message OCPP la place **offline** ;
4. si la communication est valide, les états OCPP les plus récents des connecteurs déterminent l'état opérationnel.

Table 1.1 – Règles métier de projection de la disponibilité

Preuve observée	État de la station	Interprétation métier
Au moins un connecteur Available	Disponible	Une nouvelle recharge peut être initiée.
Aucun connecteur libre et au moins un connecteur actif	Occupée	La station communique, mais tous les points utilisables sont engagés.
Aucun connecteur libre et au moins une réservation	Réservée	Les ressources disponibles sont réservées.
Aucun connecteur utilisable et présence d'un défaut	En défaut	Une intervention ou un diagnostic est nécessaire.
Station connectée sans état exploitable	Non disponible	La plateforme attend une preuve de statut cohérente.
Connexion fermée ou absence de message pendant 90 secondes	Hors ligne	La communication avec le CSMS n'est plus considérée fiable.

Le heartbeat négocié est de 30 secondes ; le seuil de 90 secondes correspond donc à trois périodes manquées. Tout autre message OCPP plus récent constitue également une preuve de vie. Les transitions sont historisées afin de permettre l'audit, et les alertes de perte de communication ou de défaut utilisent une clé de déduplication : elles sont rouvertes en cas de récurrence et résolues automatiquement lorsque la condition disparaît.

1.2 Problématique

La difficulté principale ne consiste pas à dessiner une carte ou à stocker une colonne **status**. Une borne peut être physiquement alimentée mais déconnectée du système, connectée mais sans statut récent, disponible au niveau station mais en défaut sur un connecteur, ou entièrement occupée. Une valeur ancienne conservée en base peut alors induire l'utilisateur en erreur.

Cette incertitude affecte plusieurs acteurs :

- le client risque de se déplacer vers une borne inutilisable ;
- l'opérateur peut détecter trop tard une perte de communication ;
- le technicien peut recevoir une intervention sans historique ni contexte ;
- l'administrateur manque d'indicateurs fiables pour piloter son organisation ;
- le super administrateur doit contrôler l'isolation des organisations et le fonctionnement global sans accéder arbitrairement à leurs données métier.

À ces contraintes s'ajoutent l'hétérogénéité des équipements, la gestion de plusieurs organisations, la tarification, les autorisations, la traçabilité des actions et la nécessité de tester sans disposer en permanence d'une borne physique ou d'un prestataire de paiement réel.

Problématique centrale

Comment concevoir une plateforme web multi-rôle capable de visualiser et de suivre de manière fiable la disponibilité d'un réseau de bornes, en transformant les événements OCPP en états métier, alertes et parcours opérationnels, tout en restant testable par simulation, sécurisée et extensible vers des services réels ?

La réponse attendue doit donc relier le terrain et l'application. Elle doit éviter les données purement décoratives, expliciter l'origine de chaque état, garantir qu'un acteur n'agit que dans son périmètre et couvrir un scénario complet jusqu'au paiement et à la production d'une preuve de transaction.

1.3 Étude de l'existant

1.3.1 Solutions existantes

1.3.1.1 ChargePoint

ChargePoint propose une plateforme commerciale de gestion de recharge incluant la visibilité en temps réel, des tableaux de bord, des rapports, la gestion de l'accès et des prix ainsi que la prise en charge de matériels compatibles OCPP. Son approche illustre l'intérêt d'unifier opérations, politiques d'accès et analyse dans une même interface [7]. Elle constitue toutefois un service propriétaire complet : son objectif n'est pas de

servir de base pédagogique modifiable pour étudier les règles internes de projection de disponibilité.

1.3.1.2 Driivz

Driivz se positionne comme une plateforme de gestion de réseaux de recharge à grande échelle. Sa documentation présente la surveillance en temps réel, les alertes, la gestion des incidents, l'automatisation de certaines corrections, la facturation, les services conducteurs et des capacités multi-tenant [8]. Cette couverture fonctionnelle confirme que disponibilité, maintenance et commerce ne doivent pas être conçus comme des modules isolés.

1.3.1.3 Monta

Monta Hub fournit aux opérateurs un environnement de gestion des points de recharge avec suivi d'état, contrôle d'accès, configuration des prix, automatisation opérationnelle et rapports [9]. La solution met l'accent sur l'expérience des équipes et des conducteurs. Elle sert de référence pour la simplicité des parcours, mais reste une offre commerciale dont les règles internes et les adaptations académiques ne sont pas directement maîtrisables.

1.3.1.4 CitrineOS

CitrineOS est un CSMS Open Source porté par LF Energy. Il prend en charge OCPP 1.6 et OCPP 2.0.1 et fournit une base modulaire pour le provisionnement, le contrôle, les transactions, le suivi de disponibilité et la consommation [10]. Il est particulièrement pertinent pour comprendre l'architecture d'un backend OCPP extensible. En revanche, il ne remplace pas à lui seul le produit métier, les espaces utilisateurs, la politique multi-organisation et les parcours UX propres au sujet.

1.3.2 Standards, outils et simulateurs

1.3.2.1 Référentiel OCPP et sécurité

La documentation de l'Open Charge Alliance fournit la référence fonctionnelle du protocole, ses scénarios de conformité et des recommandations de sécurité. Pour OCPP 1.6, la quatrième édition du livre blanc de sécurité couvre notamment l'établissement de connexions sécurisées, la journalisation d'événements et la mise à jour sécurisée du firmware [11]. Ces éléments justifient l'authentification des stations, le stockage d'un

secret sous forme de condensat, la traçabilité des commandes et la séparation entre environnement local et déploiement.

1.3.2.2 Simulation des stations

Le simulateur *e-mobility-charging-stations-simulator* publié par SAP est un outil Open Source en TypeScript destiné à simuler des stations OCPP-J 1.6 et 2.0 [12]. Il permet de reproduire la connexion, les heartbeats, les notifications de statut et les transactions sans borne physique. Dans le projet, il est utilisé comme source externe d'événements ; la plateforme ne modifie pas artificiellement les états en base pour faire croire à une connexion réelle.

1.3.2.3 Cartographie et paiement

La cartographie repose sur Leaflet et les données OpenStreetMap. Ce choix permet d'afficher les stations, de sélectionner des coordonnées sur la carte et de proposer un itinéraire externe sans dépendre d'un composant propriétaire unique [13, 14].

Le paiement est encapsulé derrière un adaptateur. Le MVP utilise un fournisseur simulé externe et traite sa réponse ainsi que ses webhooks comme le ferait une intégration réelle. Cette séparation permet de tester les succès, refus, délais et doublons sans transaction financière, puis d'ajouter ultérieurement un adaptateur pour un prestataire compatible avec le marché ciblé.

1.3.3 Synthèse comparative

Le tableau 1.2 synthétise le positionnement des solutions à partir de leurs fonctions publiquement documentées. Il ne constitue pas un benchmark commercial exhaustif ; il met en évidence les critères utiles au périmètre du stage.

Table 1.2 – Comparaison synthétique des solutions étudiées

Solution	Nature	Supervision et OCPP	Parcours métier	Positionnement
ChargePoint	Plateforme commerciale	Visibilité temps réel et matériel OCPP	Accès, prix, tableaux de bord et rapports	Produit mature, mais fonctionnement interne propriétaire.
Driivz	Plateforme commerciale	Monitoring, alertes, incidents et automatisation	Conducteurs, facturation, analytique et multi-tenant	Couverture large adaptée aux grands réseaux.
Monta Hub	Plateforme commerciale	État des points, disponibilité et opérations	Accès, tarification, automatisation et rapports	Forte orientation expérience opérateur et conducteur.
CitrineOS	CSMS Open Source	OCPP 1.6/2.0.1, contrôle et transactions	Briques backend à intégrer dans un produit	Base technique extensible, sans UX métier imposée.
ChargeTrackr	Prototype de stage	OCPP 1.6J, simulation, projection et alertes	Cinq espaces métier, recharge et paiement simulé	Chaîne verticale maîtrisée et adaptée au contexte pédagogique.

L'étude montre qu'un produit utile doit réunir deux qualités. Premièrement, la vérité opérationnelle doit provenir du protocole et être explicable. Deuxièmement, cette vérité doit être présentée différemment selon l'objectif de chaque acteur. C'est ce double principe qui guide la solution proposée.

1.4 Solution proposée

1.4.1 Périmètre fonctionnel

ChargeTrackr est une application web de visualisation, de supervision et de gestion d'infrastructures de recharge électrique. Elle associe un frontend React, une API REST Laravel, une base PostgreSQL, une passerelle OCPP et des services asynchrones. La plateforme couvre :

- un espace public avec présentation, carte, demande de démonstration, inscription client et authentification locale ou Google ;
- l'activation par invitation des administrateurs, opérateurs et techniciens ;
- la gestion des organisations, utilisateurs, stations, connecteurs, tarifs et abonnements ;
- la réception des messages OCPP, le calcul de disponibilité, l'historique des transitions, les alertes et les commandes distantes ;

- le parcours client de sélection, connexion, autorisation, recharge, arrêt, paiement simulé et consultation du reçu ;
- les interventions, la maintenance, les notifications, les tableaux de bord, les rapports et les exports adaptés aux rôles.

1.4.2 Espaces authentifiés par rôle

L'interface et les autorisations sont spécialisées afin d'éviter un tableau de bord générique :

Super administrateur

supervise la plateforme, les organisations, les demandes de démonstration, les paramètres globaux, les intégrations et l'audit.

Administrateur

pilote une seule organisation, ses employés, ses clients, ses actifs de recharge, sa tarification, son abonnement commercial et ses rapports.

Opérateur

surveille les stations de son organisation, traite les alertes, commande les équipements et assigne les interventions.

Technicien

consulte les stations concernées, exécute les interventions, documente les opérations avant/après et met à jour leur avancement.

Client

recherche une station, démarre et suit ses sessions, paie, consulte ses reçus et gère ses abonnements aux réseaux.

Les employés appartiennent à une seule organisation. Un client reste un utilisateur indépendant et peut adhérer à plusieurs réseaux sans dupliquer son compte. Cette distinction réduit l'ambiguïté entre identité globale et relation commerciale.

1.4.3 Chaîne verticale de fonctionnement

Le cœur du MVP peut être résumé par la chaîne suivante :

Station simulée → OCPP → passerelle → événements → disponibilité
→ carte et alertes → session → tarification → paiement simulé → reçu

Chaque étape conserve une responsabilité précise. La passerelle traduit le protocole en événements applicatifs ; le backend applique les règles métier et persiste les transitions ; le frontend consomme l'API et les mises à jour temps réel. Le paiement et la messagerie

sont traités comme des intégrations remplaçables. Cette organisation facilite les tests et évite que l'interface contienne des règles critiques.

1.4.4 Périmètre du MVP et extensions

Le MVP vise une démonstration crédible avec OCPP 1.6J, un simulateur de station et un paiement simulé. Il ne prétend pas certifier un matériel, traiter de l'argent réel ni assurer immédiatement une exploitation industrielle. L'intégration d'une borne physique, d'un prestataire financier réel, d'OCPP 2.0.1, de la gestion distante du firmware ou d'une prédiction de panne par intelligence artificielle constitue une évolution ultérieure. La priorité est d'abord donnée à la cohérence du domaine, à la traçabilité et à la sécurité du parcours principal.

1.5 Méthodologie de travail

1.5.1 Approche itérative et incrémentale

Le projet est réalisé individuellement. Une application stricte de Scrum, qui suppose des responsabilités et des événements pensés pour une équipe, ne serait donc pas adaptée. La démarche retenue est **itérative et incrémentale, inspirée des principes agiles**. Le Scrum Guide est utilisé comme référence pour la transparence, l'inspection et l'adaptation, sans revendiquer l'ensemble du cadre Scrum [15].

Le backlog produit fournit une vision globale et des user stories estimées. Les dépendances ne sont pas ignorées : les incréments sont ordonnés de manière à construire progressivement une chaîne verticale utilisable. Un incrément peut dépendre d'une fondation antérieure tout en produisant un résultat observable et testable.

Table 1.3 – Organisation des incréments de réalisation

Incrément	Objectif principal	Résultat vérifiable
Cadrage	Acteurs, backlog, cas d'utilisation, architecture et prototype	Périmètre partagé et parcours validables.
Fondations	Authentification, invitations, rôles et isolation	Accès sécurisé selon le rôle et l'organisation.
Réseau	Stations, connecteurs, carte, OCPP et disponibilité	État calculé à partir d'une station simulée.
Recharge	Parcours client, transaction, mesures et paiement	Session complète jusqu'au reçu.
Exploitation	Alertes, interventions, maintenance et notifications	Incident traçable de sa détection à sa résolution.
Pilotage	Tableaux de bord, commerce, rapports et exports	Indicateurs et documents adaptés à chaque acteur.
Stabilisation	Sécurité, tests, déploiement et documentation	Produit démontrable, contrôlé et transmissible.

1.5.2 Cycle de traitement d'un bloc

Avant chaque bloc, le périmètre, les règles, les sources externes et les critères d'acceptation sont explicités. Après validation, le travail suit généralement les étapes suivantes :

1. analyse des user stories et des règles métier concernées ;
2. conception du parcours et des contrats API ;
3. implémentation backend et frontend en parallèle lorsque les contrats sont stabilisés ;
4. tests automatisés ciblés, compilation et vérification visuelle ;
5. démonstration du scénario, correction des retours puis versionnement Git.

Cette organisation réduit le risque de développer longtemps sur une hypothèse incorrecte. Elle permet également de faire évoluer le backlog lorsqu'un besoin technique important apparaît, comme l'intégration d'un simulateur OCPP ou le calcul des états à partir des heartbeats.

1.5.3 Traçabilité et contrôle de la qualité

Le dépôt Git constitue la référence du code source et de la documentation. Les changements sont regroupés dans des commits cohérents et publiés sur le dépôt distant. Les règles sensibles sont couvertes par des tests unitaires ou d'intégration, notamment la projection de disponibilité, l'isolation des organisations, les autorisations, les webhooks et les transitions de workflow. Les parcours majeurs font aussi l'objet de tests manuels scénarisés avec les services nécessaires : API Laravel, worker de file, serveur WebSocket, passerelle et simulateur OCPP.

La qualité visuelle est vérifiée dans plusieurs tailles d'écran, mais elle reste subordonnée à la vérité fonctionnelle : un graphique ou un indicateur ne doit pas présenter une donnée synthétique comme si elle provenait du terrain. Cette règle guide également la documentation et les démonstrations du projet.

Conclusion

Ce chapitre a présenté le contexte du stage, les acteurs et mécanismes fondamentaux d'un réseau de recharge, ainsi que la difficulté de transformer des messages techniques en une disponibilité métier fiable. L'étude de ChargePoint, Driivz, Monta et CitrineOS montre l'importance d'unifier supervision, expérience client et opérations, tandis que les références OCPP encadrent les échanges avec les bornes. ChargeTrackr retient une architecture testable par simulation, des espaces spécialisés par rôle et une démarche incrémentale. Le chapitre suivant formalise les besoins fonctionnels et non fonctionnels qui découlent de ce cadrage.

Spécification et analyse des besoins

Introduction

Ce chapitre transforme le cahier des charges en un périmètre vérifiable. Il identifie les acteurs humains et externes, formalise les besoins fonctionnels et non fonctionnels, puis présente le Product Backlog révisé et son découpage en incréments. Enfin, un diagramme global et huit scénarios détaillés décrivent les interactions les plus risquées ou structurantes du produit.

2.1 Spécification des besoins

2.1.1 Identification des acteurs

Un acteur représente une entité extérieure au système qui poursuit un objectif au moyen de celui-ci. Une base de données, un worker de file ou le scheduler Laravel ne sont donc pas des acteurs : ils appartiennent à l'implémentation interne. En revanche, une borne, Google OAuth ou un prestataire de paiement sont des acteurs secondaires, car ils échangent avec la plateforme par une interface clairement délimitée.

La notion d'**utilisateur authentifié** est utilisée comme acteur abstrait pour regrouper les capacités communes : connexion, profil, paramètres personnels, onboarding et notifications. Les cinq rôles authentifiés spécialisés restent le Super Administrateur, l'Administrateur, l'Opérateur, le Technicien et le Client.

Table 2.1 – Acteurs humains de la plateforme

Acteur	Catégorie	Responsabilités et périmètre
Visiteur	Principal	Consulte la présentation publique, les informations publiques des bornes, contacte la plateforme, crée un compte client ou dépose une demande de démonstration.
Client	Principal	Utilisateur global sans organisation fixe. Il peut utiliser les bornes de plusieurs organisations, gérer ses abonnements, lancer ses propres sessions, payer et consulter uniquement ses données.
Opérateur	Principal	Employé d'une seule organisation. Il met en service les stations, supervise OCPP, traite les alertes, exécute les commandes distantes, assigne les interventions et planifie les maintenances dans son périmètre.
Technicien	Principal	Employé d'une seule organisation. Il consulte les actifs en lecture, traite uniquement les interventions et maintenances qui lui sont assignées, puis produit les preuves et rapports techniques.
Administrateur	Principal	Responsable d'une seule organisation. Il gère ses opérateurs et techniciens, ses clients observés, ses actifs, tarifs, documents, rapports et son abonnement commercial, sans administrer la plateforme globale.
Super Administrateur	Principal	Gère les organisations et leurs administrateurs, les demandes de démonstration, les offres SaaS, l'audit, les rôles, les paramètres globaux et l'état des intégrations.

Table 2.2 – Acteurs secondaires et systèmes externes

Acteur	Nature	Interaction avec ChargeTrackr
Borne ou simulateur OCPP	Système externe	Ouvre une connexion WebSocket authentifiée, émet les messages OCPP, reçoit les commandes distantes et retourne leur résultat.
Google OAuth	Fournisseur d'identité	Authentifie un compte Google vérifié sans transmettre son mot de passe à la plateforme.
Service de paiement	Service externe	Autorise, capture ou libère un montant et envoie des webhooks signés. Dans le MVP, ce service est simulé mais reste hors du domaine métier.
Service email	Service externe	Distribue les vérifications, invitations, réinitialisations et notifications transactionnelles placées dans la file.
Service cartographique	Service externe	Fournit les tuiles et données nécessaires à l'affichage de la carte et à l'ouverture d'un itinéraire.

Clarification du rôle financier

La *finance* n'est pas modélisée comme un acteur autonome. Les décisions commerciales sont prises par le Super Administrateur ou l'Administrateur selon leur périmètre. Le seul acteur externe du paiement est le prestataire technique qui traite une transaction. Cette distinction évite de confondre une fonction métier, un rôle humain et un système tiers.

2.1.2 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit permettre de réaliser. Ils sont regroupés par acteur, mais les contrôles d'autorisation restent appliqués côté API même lorsqu'une action n'est pas affichée dans l'interface.

2.1.2.1 Fonctions communes aux utilisateurs authentifiés

Code	Besoin fonctionnel
BF-01	Se connecter par email et mot de passe ou, lorsque le compte le permet, par Google OAuth ; se déconnecter et récupérer son accès.
BF-02	Consulter et modifier les données personnelles autorisées, l'adresse, l'avatar et les liens professionnels pertinents.
BF-03	Configurer le fuseau horaire, le rayon de recherche, les préférences de notification et le mot de passe lorsqu'un mot de passe local existe.
BF-04	Suivre un onboarding court adapté au rôle, puis retrouver des indications contextuelles sans être bloqué dans une visite générique.
BF-05	Consulter les notifications personnelles, leur contexte, leur statut lu ou non lu et marquer un ensemble cohérent comme lu.
BF-06	Utiliser une recherche globale limitée aux objets réellement accessibles.

2.1.2.2 Super Administrateur

Code	Besoin fonctionnel
BF-07	Qualifier, rejeter ou convertir une demande de démonstration en organisation d'essai et invitation d'administrateur.
BF-08	Créer, consulter, modifier, activer ou suspendre les organisations et gérer leurs administrateurs.
BF-09	Consulter les indicateurs globaux, l'audit filtrable et la matrice des rôles et permissions.
BF-10	Consulter l'état des intégrations sans exposer leurs secrets et modifier uniquement les paramètres globaux autorisés.
BF-11	Gérer les offres SaaS, essais, capacités, abonnements et factures des organisations, avec historique des transitions.
BF-12	Produire les analyses et exports globaux de la plateforme.

2.1.2.3 Administrateur d'organisation

Code	Besoin fonctionnel
BF-13	Inviter, modifier, désactiver ou supprimer les opérateurs et techniciens de sa seule organisation, sans connaître leur mot de passe.
BF-14	Consulter les clients ayant réellement utilisé les bornes de l'organisation, sans modifier leur compte global ni leurs activités chez d'autres opérateurs.
BF-15	Administrer les stations, connecteurs, commandes, alertes, interventions, maintenances et documents de son organisation.
BF-16	Définir les tarifs, plans clients et affectations applicables aux stations ou connecteurs de son organisation.
BF-17	Consulter les indicateurs métier, produire et échanger des rapports et exporter les données autorisées.
BF-18	Suivre l'offre commerciale, les quotas, les échéances et factures de son organisation et demander un changement d'offre.

2.1.2.4 Opérateur

Code	Besoin fonctionnel
BF-19	Mettre en service une station et ses connecteurs de manière atomique, la positionner sur la carte et préparer sa liaison à une borne physique ou simulée.
BF-20	Consulter les stations en vue carte, liste ou tableau avec disponibilité calculée, connexion, dernière preuve de vie et alertes.
BF-21	Examiner une station, sa télémétrie vérifiée, ses connecteurs, événements, commandes, sessions, maintenances, alertes et documents.
BF-22	Exécuter les commandes OCPP autorisées : redémarrage logiciel, déverrouillage et changement de disponibilité pour maintenance.
BF-23	Filtrer, acquitter, assigner, suivre ou annuler une alerte dans le respect de son cycle de vie et de son SLA.

BF-24	Créer et suivre les interventions et maintenances préventives ou correctives, y compris leur récurrence et leur calendrier.
BF-25	Superviser les sessions et paiements de l'organisation sans agir à la place d'un client sur ses données privées.
BF-26	Consulter et échanger des rapports opérationnels adaptés à son rôle.

2.1.2.5 Technicien

Code	Besoin fonctionnel
BF-27	Consulter en lecture les stations, connecteurs, cartes et informations techniques de son organisation.
BF-28	Consulter les alertes, interventions et maintenances qui lui sont assignées, avec priorité, échéance et historique.
BF-29	Faire progresser une intervention selon les transitions autorisées, ajouter des notes et téléverser des preuves avant/après privées.
BF-30	Produire un rapport final comprenant diagnostic, actions, pièces, contrôles terrain et résultat, puis demander un suivi lorsque nécessaire.
BF-31	Consulter ses propres indicateurs de terrain et échanger des rapports avec les destinataires autorisés de son organisation.

2.1.2.6 Client

Code	Besoin fonctionnel
BF-32	Rechercher les stations publiques des organisations actives, utiliser la géolocalisation selon un rayon choisi et filtrer les résultats.
BF-33	Consulter les connecteurs et le tarif effectif, copier les coordonnées ou ouvrir un itinéraire vers la station sélectionnée.
BF-34	Démarrer une recharge guidée : choisir station et connecteur, brancher le câble, attendre sa détection OCPP, autoriser le paiement puis lancer la commande.

-
- | | |
|-------|--|
| BF-35 | Suivre en temps réel la durée, l'énergie, la puissance et le coût provisoire, puis arrêter la session manuellement ou par une limite configurée. |
| BF-36 | Consulter l'historique multi-organisations de ses sessions et paiements, prévisualiser ou télécharger un reçu. |
| BF-37 | Consulter les plans proposés par les organisations et gérer plusieurs abonnements indépendants avec leurs factures. |
| BF-38 | Utiliser un identifiant OCPP virtuel ; préparer l'évolution vers une carte RFID ou la lecture d'un QR code physique sans exposer de secret. |
-

2.1.2.7 Visiteur et traitements automatiques

Code	Besoin fonctionnel
BF-39	Présenter la solution et les moyens de contact, permettre l'inscription client et la demande de démonstration sans exposer les données protégées.
BF-40	Recevoir, authentifier et rendre idempotents les événements OCPP, puis projeter les états métier et leurs transitions.
BF-41	Détecter les absences de communication, défauts, échéances SLA ou maintenances proches et produire les alertes et notifications correspondantes.
BF-42	Traiter en tâche de fond les emails, occurrences récurrentes, contrôles de SLA, exports volumineux et événements de paiement.
BF-43	Vérifier les signatures et l'idempotence des webhooks de paiement, puis réconcilier le paiement, la session et le reçu.

2.1.3 Besoins non fonctionnels

Les objectifs du cahier des charges sont transformés en exigences vérifiables. Les valeurs de production qui ne peuvent pas être garanties par un environnement local sont explicitement présentées comme des objectifs à valider lors du déploiement.

Table 2.10 – Exigences non fonctionnelles

Code	Qualité	Exigence et critère de vérification
ENF-01	Performance	Sous charge nominale et avec pagination, une page courante doit devenir utilisable en moins de deux secondes ; les opérations longues sont asynchrones et signalent leur progression.
ENF-02	Temps réel	Un événement OCPP accepté doit mettre à jour sa projection et être diffusé sans rechargement manuel. Une perte de communication est détectée après 90 secondes sans message, conformément à la règle métier retenue.
ENF-03	Disponibilité du service	La cible de production est de 99,9 % hors maintenance planifiée. Cette cible doit être mesurée par supervision après déploiement ; elle n'est pas déclarée atteinte par le seul MVP local.
ENF-04	Sécurité des accès	Toute route protégée utilise une session Sanctum, la protection CSRF, RBAC et une policy sur la ressource. Un identifiant deviné d'une autre organisation ne doit retourner aucune donnée.
ENF-05	Isolation multi-tenant	Un administrateur, opérateur ou technicien appartient à exactement une organisation. Toutes les lectures, écritures, diffusions et exports sont limités à ce périmètre.
ENF-06	Protection des secrets	Aucun secret OAuth, email, paiement ou OCPP n'est inclus dans le frontend, les logs applicatifs ou une réponse API. Les mots de passe et jetons persistés sont condensés ou chiffrés selon leur usage.
ENF-07	Intégrité et idempotence	Une station et ses connecteurs sont créés atomiquement. Les événements OCPP, webhooks, commandes et tâches récurrentes supportent la répétition sans doublon métier.

Code	Qualité	Exigence et critère de vérification
ENF-08	Traçabilité	Les changements sensibles conservent acteur, date, ancienne et nouvelle valeur ou événement métier. Les rapports finaux et reçus restent reproductibles.
ENF-09	Maintenabilité	Les contrôleurs délèguent les règles aux services et polices ; les contrats frontend sont centralisés ; les modules critiques sont couverts par des tests automatisés et documentés.
ENF-10	Extensibilité	Les fournisseurs de paiement, email, cartographie et identité sont intégrés derrière des contrats ou configurations remplaçables. OCPP reste séparé de l'API métier.
ENF-11	Ergonomie	Les parcours critiques utilisent des étapes courtes, des états de chargement et des erreurs actionnables. L'interface reste cohérente entre rôles sans rendre leurs tableaux de bord identiques.
ENF-12	Responsive et compatibilité	Les parcours principaux sont utilisables sur ordinateur et tablette, et les parcours client essentiels sur mobile. Les dernières versions de Chrome, Edge et Firefox sont ciblées.
ENF-13	Confidentialité	La position courante du client reste dans l'état transitoire du navigateur et n'est pas persistée. Les pièces d'intervention privées sont servies après autorisation.
ENF-14	Portabilité	L'environnement peut être reproduit avec Git, PHP, Node.js, PostgreSQL et Docker sous WSL2 pour les services simulés, sans dépendre d'un poste unique.
ENF-15	Interopérabilité	Les échanges métier utilisent une API REST JSON documentable par OpenAPI ; les stations utilisent OCPP 1.6 JSON sur WebSocket et le paiement suit un contrat fournisseur indépendant.

2.2 Planification du projet

2.2.1 Méthode de priorisation et d'estimation

Deux informations distinctes sont associées à chaque user story :

- la **priorité** exprime la valeur métier et l'urgence. Les niveaux Haute, Moyenne et Faible correspondent respectivement à une adaptation des catégories *Must*, *Should* et *Could* de MoSCoW ;
- les **story points** estiment l'effort relatif, l'incertitude et le risque selon la suite de Fibonacci : 1, 2, 3, 5, 8 et 13.

Les points ne représentent donc ni des jours ni une priorité. Une fonctionnalité hautement prioritaire peut recevoir peu de points si elle est simple, tandis qu'une évolution facultative comme le firmware peut recevoir 13 points en raison de son risque technique.

2.2.2 Product Backlog révisé

Le backlog initial de 30 user stories a été enrichi à mesure que les relations multi-organisations, la disponibilité calculée et les parcours réels ont été clarifiés. La gestion des véhicules, retirée du produit après validation UX, est remplacée par un onboarding adapté au rôle. Deux user stories distinguent désormais la gestion commerciale globale de la consultation de l'abonnement par l'administrateur. Le backlog révisé contient 42 user stories pour 301 points.

Table 2.11 – Product Backlog de ChargeTrackr

ID	User Story	Priorité	Pts
US-01	En tant que visiteur, je veux créer un compte client ou me connecter de manière sécurisée par email ou Google.	Haute	8
US-02	En tant qu'utilisateur authentifié, je veux consulter et mettre à jour mon profil ainsi que les paramètres de sécurité de mon compte.	Moyenne	3
US-03	En tant que super administrateur, je veux gérer les organisations, leurs administrateurs et les accès globaux de la plateforme.	Haute	8
US-04	En tant qu'administrateur, je veux créer, modifier et désactiver les comptes des opérateurs et techniciens de mon organisation.	Haute	5

ID	User Story	Priorité	Pts
US-05	En tant que super administrateur, je veux consulter l'audit global, l'état des intégrations et les statistiques d'utilisation de la plateforme.	Moyenne	5
US-06	En tant qu'administrateur, je veux consulter les clients ayant utilisé les bornes de mon organisation avec leurs sessions et statistiques.	Moyenne	5
US-07	En tant qu'opérateur, je veux ajouter, modifier et désactiver les bornes de mon organisation avec leurs informations techniques et géographiques.	Haute	8
US-08	En tant qu'opérateur, je veux gérer les connecteurs associés aux bornes de mon organisation avec leur type, puissance et état.	Haute	5
US-09	En tant qu'opérateur, je veux consulter la liste des bornes avec leur disponibilité calculée, leur connexion et leur dernière activité.	Haute	8
US-10	En tant qu'opérateur, je veux visualiser les bornes sur une carte et ajouter une borne en sélectionnant son emplacement.	Haute	8
US-11	En tant que technicien, je veux consulter sans modification les bornes et leur emplacement sur la carte de mon organisation.	Haute	5
US-12	En tant que client, je veux rechercher et filtrer les bornes publiques selon leur disponibilité, connecteur et tarif.	Haute	8
US-13	En tant qu'opérateur, je veux consulter la fiche détaillée d'une borne avec ses connecteurs, sessions, historique et alertes.	Haute	8
US-14	En tant que plateforme, je veux recevoir et valider automatiquement les messages OCPP 1.6 JSON des bornes physiques ou simulées.	Haute	13
US-15	En tant que plateforme, je veux calculer l'état des bornes à partir des événements OCPP, transactions, heartbeats et délais de non-réponse.	Haute	13
US-16	En tant qu'opérateur, je veux superviser en temps réel les états disponible, occupée, non disponible, maintenance, déconnectée ou en défaut.	Haute	13
US-17	En tant qu'opérateur, je veux consulter un tableau de bord avec disponibilité, énergie, sessions actives et taux d'indisponibilité.	Haute	8
US-18	En tant qu'administrateur, je veux consulter le tableau de bord de mon organisation avec utilisateurs, clients, revenus, régions et bornes.	Haute	8
US-19	En tant qu'opérateur, je veux recevoir des alertes lorsqu'une borne est déconnectée, en panne, en surchauffe ou en erreur de communication.	Haute	8
US-20	En tant qu'opérateur, je veux filtrer, suivre et assigner les alertes selon leur gravité, statut et borne.	Moyenne	5
US-21	En tant que technicien, je veux consulter les alertes et interventions qui me sont assignées.	Haute	5

ID	User Story	Priorité	Pts
US-22	En tant que technicien, je veux créer un rapport d'intervention avec diagnostic, actions, pièces et photos.	Moyenne	8
US-23	En tant qu'opérateur, je veux planifier et suivre les maintenances préventives et correctives.	Moyenne	8
US-24	En tant que client, je veux démarrer une session sur une borne disponible, quelle que soit son organisation.	Haute	8
US-25	En tant que client, je veux arrêter ma session et consulter le résumé de durée, énergie et coût.	Haute	5
US-26	En tant qu'utilisateur autorisé, je veux consulter l'historique des sessions de mon périmètre avec durée, énergie, coût et borne.	Moyenne	5
US-27	En tant que nouvel utilisateur, je veux un onboarding adapté à mon rôle afin de configurer mon espace et atteindre une première action utile.	Moyenne	5
US-28	En tant que client, je veux utiliser une carte RFID ou un QR code pour m'identifier et lancer une recharge.	Moyenne	8
US-29	En tant que client, je veux payer mes sessions, suivre le paiement et prévisualiser ou télécharger mes reçus.	Moyenne	8
US-30	En tant que plateforme, je veux traiter les paiements au moyen d'un adaptateur simulé et remplaçable.	Moyenne	8
US-31	En tant qu'administrateur, je veux configurer les tarifs selon le kWh, la durée ou les plages horaires et les affecter aux actifs.	Moyenne	8
US-32	En tant que client, je veux choisir les plans des organisations et gérer simultanément plusieurs abonnements.	Moyenne	8
US-33	En tant qu'utilisateur autorisé, je veux consulter des analyses adaptées à mon rôle et échanger des rapports opérationnels.	Moyenne	8
US-34	En tant qu'utilisateur autorisé, je veux exporter les données et rapports de mon périmètre en PDF, JSON ou CSV.	Faible	5
US-35	En tant qu'administrateur, je veux gérer les paramètres locaux de mon organisation : TVA, logo, langue, fuseau et notifications.	Faible	5
US-36	En tant que super administrateur, je veux configurer les intégrations globales : paiement, notifications, cartographie, OAuth2, OCPP et sécurité.	Faible	8
US-37	En tant qu'administrateur, je veux gérer les contrats, garanties, notices et plans techniques liés aux bornes.	Faible	5
US-38	En tant qu'administrateur, je veux gérer les mises à jour firmware des bornes à distance.	Faible	13

ID	User Story	Priorité	Pts
US-39	En tant que visiteur, je veux consulter la présentation, contacter la plateforme et envoyer une demande de démonstration.	Moyenne	3
US-40	En tant qu'utilisateur authentifié, je veux recevoir des notifications in-app et email et gérer mes préférences.	Moyenne	5
US-41	En tant que super administrateur, je veux gérer les offres SaaS, essais, abonnements et factures des organisations.	Haute	8
US-42	En tant qu'administrateur, je veux suivre l'abonnement commercial, les limites et factures de mon organisation et demander un changement d'offre.	Moyenne	5
		Total	301 points

Table 2.12 – Répartition du backlog par priorité

Priorité	User stories	Points	Part des points
Haute	20	160	53,2 %
Moyenne	17	105	34,9 %
Faible	5	36	12,0 %
Total	42	301	100 %

2.2.3 Découpage en incréments

Un incrément n'est pas synonyme de sprint. Il correspond ici à un ensemble cohérent de capacités produisant un résultat démontrable. Les incréments sont dépendants mais ordonnés : le suivant peut s'appuyer sur les fondations validées du précédent.

Table 2.13 – Backlogs des incréments

Inc.	Thème	User stories principales	Résultat attendu
INC-01	Entrée, identité et multi-tenant	US-01 à US-04, US-06, US-27, US-39	Un visiteur peut entrer dans le produit ; chaque rôle obtient un espace protégé et une organisation correctement isolée.
INC-02	Stations et cartographie	US-07 à US-13, US-37	Les actifs sont mis en service, localisés, consultables selon le rôle et accompagnés de leurs documents.
INC-03	OCCP et disponibilité	US-14 à US-16, US-19	Une station simulée communique réellement ; ses événements produisent des états, transitions et alertes vérifiables.
INC-04	Exploitation terrain	US-20 à US-23, US-40	Une anomalie est qualifiée, assignée, traitée, documentée et notifiée jusqu'à sa résolution.
INC-05	Recharge et paiement	US-24 à US-26, US-28 à US-30	Le client réalise une session complète avec mesures, arrêt, paiement simulé et reçu.
INC-06	Tarification et commerce	US-31, US-32, US-41, US-42	Les prix et abonnements clients sont séparés du contrat SaaS de l'organisation, avec limites et factures propres.

Inc.	Thème	User stories principales	Résultat attendu
INC-07	Pilotage et paramétrage	US-05, US-17, US-18, US-33 à US-36	Chaque rôle dispose d'analyses pertinentes ; rapports, audit, intégrations et paramètres deviennent exploitables.
INC-08	Industrialisation et évolutions	US-34, US-38	Les exports sont généralisés ; le firmware reste planifié après validation matérielle et sécuritaire.

2.2.4 Matrice de traçabilité fonctionnelle

La matrice suivante relie les grands groupes de besoins à leurs principaux contrats API et écrans. Les routes détaillées et la spécification OpenAPI seront présentées dans les livrables techniques.

Table 2.14 – Traçabilité entre backlog, API et interfaces

Domaine	User stories	Contrats API principaux	Interfaces principales
Identité et entrée	US-01–06, 27, 39	Auth, profil, onboarding, utilisateurs, organisations, demandes de démo	Landing, connexion, activation, bienvenue, profil, utilisateurs, organisations
Stations et carte	US-07–13, 37	Stations, commissioning, connecteurs, carte, télé-métrie, documents	Stations, recherche de station, détail station, cartes par rôle
OCPD et disponibilité	US-14–16, 19	API interne OCPD, commandes, transitions, alertes	Détail station, supervision, carte, centre d'alertes

Domaine	User stories	Contrats API principaux	Interfaces principales
Exploitation	US-20–23, 40	Alertes, interventions, maintenances, notifications	Alertes, interventions, maintenance, calendrier, cloche de notifications
Recharge et paiement	US-24–30	Tentatives, sessions, paiements, reçu, web-hook fournisseur	Assistant de recharge, sessions, vue live, paiements et reçus
Tarification et commerce	US-31, 32, 41, 42	Tarifs, plans, abonnements clients, portefeuille commercial, factures	Tarifs, abonnements, portefeuille commercial, facturation organisationnelle
Pilotage et rapports	US-05, 17–18, 33–36	Dashboards, reporting, rapports internes, exports, audit, intégrations, paramètres	Tableaux de bord par rôle, analyses, rapports, audit, intégrations, paramètres

2.3 Analyse globale des cas d'utilisation

2.3.1 Diagramme global

Le diagramme global de la figure 2.1 regroupe les capacités communes et les fonctions propres à chaque rôle. Il distingue les acteurs humains, la borne de recharge et le service de paiement, puis explicite les dépendances **include**, l'extension liée à l'assignation d'une intervention et les généralisations d'acteurs. Les scénarios détaillés complètent cette vue en précisant les préconditions, les contrôles d'autorisation et les cas d'erreur.

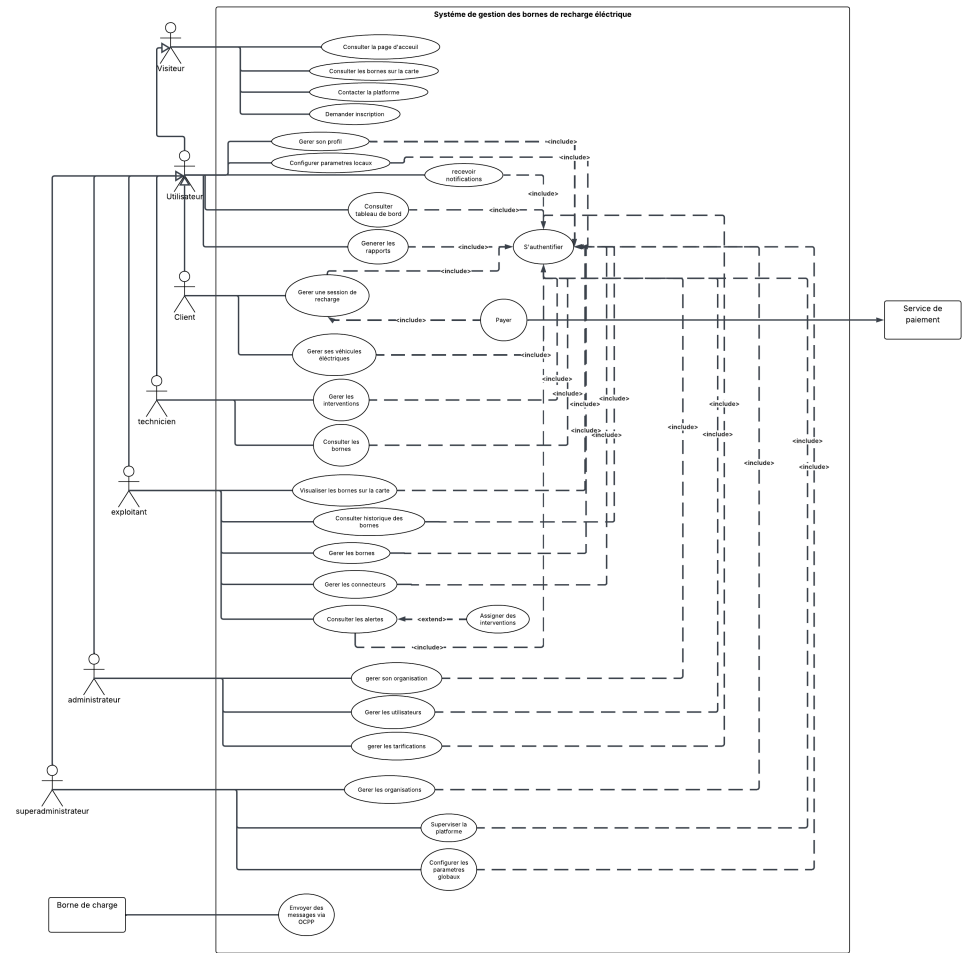


Figure 2.1 – Diagramme global des cas d'utilisation de ChargeTrackr

Conventions du diagramme

Les acteurs humains sont représentés par des silhouettes, les systèmes externes par des rectangles et les fonctionnalités par des ellipses placées dans la frontière du système. Une ligne continue exprime une association ; une flèche en pointillés précise une relation **include** ou **extend**. Les contrôles de permission, l'organisation et les scénarios d'erreur sont détaillés ci-dessous.

2.3.2 Catalogue des cas majeurs

Table 2.15 – Catalogue des cas d'utilisation majeurs

ID	Cas d'utilisation	Acteur principal	Résultat métier
UC-01	Demander une démonstration et provisionner une organisation	Visiteur, Super Administrateur	Une organisation d'essai et son administrateur invité sont créés après décision explicite.
UC-02	Inviter et activer un employé	Administrateur	Un opérateur ou technicien définit lui-même son mot de passe et rejoint la bonne organisation.
UC-03	Mettre en service une station OCPP	Opérateur	La station, ses connecteurs et son identité OCPP sont créés atomiquement et prêts à être reliés au simulateur ou au matériel.
UC-04	Calculer la disponibilité	Borne / plate-forme	Les preuves OCPP deviennent un état métier, une transition et, si nécessaire, une alerte.
UC-05	Effectuer une recharge et payer	Client	Une transaction technique produit une session mesurée, un paiement réconcilié et un reçu.

ID	Cas d'utilisation	Acteur principal	Résultat métier
UC-06	Traiter une alerte et une intervention	Opérateur, Technicien	L'incident est qualifié, assigné, documenté et clôturé avec une chronologie.
UC-07	Gérer le cycle commercial d'une organisation	Super Administrateur, Administrateur	L'essai, l'offre, les limites et factures évoluent sans supprimer les données de l'organisation.
UC-08	Générer et échanger un rapport	Utilisateur autorisé	Une analyse adaptée au rôle est exportée ou envoyée avec pièces jointes et suivi.

2.3.3 UC-01 – Demander une démonstration et provisionner une organisation

Objectif	Convertir une demande publique qualifiée en organisation d'essai et compte administrateur sécurisé.
Acteurs	Visiteur et Super Administrateur ; service email comme acteur secondaire.
Préconditions	Les demandes de démonstration sont activées ; l'adresse n'est pas bloquée par les protections anti-abus.
Déclencheur	Le visiteur valide le formulaire public.
Postconditions	La demande est rejetée ou reliée à une organisation d'essai et à une invitation administrateur traçable.

Scénario nominal

1. Le visiteur renseigne son identité, son organisation, sa taille, ses objectifs multiples, son message et son consentement.
2. Le système valide les champs, le honeypot et la limitation de débit, puis enregistre la demande au statut *nouvelle*.
3. Un accusé de réception est envoyé sans lien vers l'espace Super Administrateur.

4. Le Super Administrateur ouvre la demande et démarre sa revue.
5. Il vérifie le contexte, ajuste la durée d'essai et saisit le nom de l'organisation et de son futur administrateur.
6. Le système crée atomiquement l'organisation, l'abonnement d'essai, le compte administrateur en attente et l'invitation.
7. L'invitation à usage unique est envoyée au demandeur.
8. Le demandeur l'accepte, définit son mot de passe, complète éventuellement le logo de l'organisation et accède à l'onboarding administrateur.

Alternatives et exceptions

- si la demande n'est pas pertinente, le Super Administrateur la rejette avec une raison ; aucun compte n'est créé ;
- si l'email ou l'organisation provoque un conflit, la transaction est annulée sans création partielle ;
- si l'invitation expire ou doit être invalidée, elle est révoquée avant qu'une nouvelle invitation ne soit émise ;
- les transitions intermédiaires sans action métier ne sont pas exposées comme des statuts sélectionnables manuellement.

2.3.4 UC-02 – Inviter et activer un employé

Objectif	Donner un accès opérateur ou technicien sans transmettre de mot de passe initial.
Acteurs	Administrateur, employé invité et service email.
Préconditions	L'administrateur et son organisation sont actifs ; la limite d'employés de l'offre commerciale n'est pas atteinte.
Déclencheur	L'administrateur choisit <i>Ajouter un employé</i> .
Postconditions	Le compte activé appartient à exactement une organisation et possède uniquement le rôle choisi.

Scénario nominal

1. L'administrateur saisit le nom, l'email, le rôle opérateur ou technicien, l'équipe et les coordonnées utiles.

2. Le backend ignore tout identifiant d'organisation venant du navigateur et impose celle de l'administrateur.
3. Un compte *pending* sans mot de passe utilisable est créé avec une invitation limitée dans le temps.
4. L'employé reçoit le lien, vérifie les informations non modifiables et choisit son mot de passe.
5. L'invitation est consommée une seule fois, le compte devient actif et l'onboarding du rôle est proposé à la première connexion.

Alternatives et exceptions

- un rôle administrateur, client ou super administrateur est refusé dans ce workflow ;
- un email déjà actif, un quota atteint ou une organisation inactive empêche l'invitation ;
- l'administrateur peut envoyer un rappel tant que le lien est valide, ou renouveler le lien après invalidation de l'ancien ;
- aucun administrateur ne peut afficher, choisir ou réinitialiser le mot de passe de l'employé.

2.3.5 UC-03 – Mettre en service une station OCPP

Objectif	Créer une station exploitable, ses connecteurs physiques et son identité de communication dans une seule opération.
Acteurs	Opérateur ou Administrateur ; borne physique ou simulateur SAP.
Préconditions	L'organisation est autorisée à ajouter une station et n'a pas atteint sa capacité commerciale.
Déclencheur	L'utilisateur ouvre l'assistant de mise en service.
Postconditions	La station est persistée avec ses connecteurs et une cible de commissioning ; son secret en clair n'est affiché qu'une fois si requis.

Scénario nominal

1. L'opérateur renseigne le nom, la référence, le fabricant, le modèle, la puissance, l'adresse et la version OCPP.
2. Il clique sur la carte ou déplace le marqueur ; latitude et longitude sont remplies automatiquement mais restent éditables.

3. Il ajoute chaque connecteur avec son libellé, son identifiant OCPP, son type, son courant et sa puissance maximale.
4. Il choisit la cible *simulateur SAP local* ou *station externe* et confirme l'identité OCPP.
5. Le système vérifie les références, identités et identifiants connecteur, puis crée l'ensemble dans une transaction.
6. Pour le simulateur local, l'interface fournit la commande d'enregistrement du profil ; pour une borne externe, elle affiche une fois les informations de connexion nécessaires.
7. La borne ouvre sa connexion, envoie `BootNotification` et `StatusNotification`, puis la fiche reflète la connexion réelle.

Alternatives et exceptions

- une référence comme CT-TUN-001 déjà utilisée doit être remplacée par une valeur unique ; elle ne peut pas désigner deux stations ;
- une erreur sur un connecteur annule la création complète ;
- tant que le simulateur n'est pas enregistré et redémarré, la station reste en attente de connexion ; la création en base ne simule pas un heartbeat ;
- une rotation de secret invalide l'ancien secret sans modifier l'identité publique de la station.

2.3.6 UC-04 – Calculer la disponibilité

Objectif	Transformer les preuves OCPP et les dérogations autorisées en état métier explicable.
Acteurs	Borne ou simulateur ; moteur de projection de la plateforme.
Préconditions	La station est provisionnée pour OCPP et ses connecteurs sont connus.
Déclencheur	Connexion, fermeture, message OCPP ou contrôle périodique de connectivité.
Postconditions	Les états courants, transitions, alertes et diffusions temps réel sont cohérents et audités.

Scénario nominal

1. La passerelle authentifie la station et vérifie la signature de l'appel interne transmis à Laravel.
2. Le message est déduplicué et persisté avant projection.
3. Le moteur applique l'ordre : désactivation manuelle, maintenance planifiée, perte de communication, puis états OCPP des connecteurs.
4. Au moins un connecteur libre rend la station disponible ; sinon les états actifs, réservés ou en défaut déterminent le résultat.
5. Toute modification d'état ou de raison crée une transition et un événement temps réel.
6. Une anomalie de communication ou de connecteur crée ou rouvre une alerte déduplicuée ; sa disparition la résout automatiquement.

Alternatives et exceptions

- après 90 secondes sans aucun message, la station devient hors ligne même si un ancien statut **Available** subsiste ;
- un message tardif ne peut pas écraser une preuve plus récente ;
- une maintenance ou désactivation manuelle reste prioritaire sur un statut disponible envoyé par la borne ;
- une station non provisionnée pour OCPP conserve son statut administratif et n'est pas artificiellement projetée.

2.3.7 UC-05 – Effectuer une recharge et payer

Objectif	Permettre au client de réaliser une recharge mesurée et de régler le montant final sans créer de session fictive.
Acteurs	Client, borne OCPP et service de paiement.
Préconditions	Client actif et vérifié, station publique connectée, connecteur disponible, tarif effectif et aucune session concurrente.
Déclencheur	Le client choisit <i>Recharger</i> sur une station.
Postconditions	La transaction est arrêtée, le montant final est capturé ou signalé impayé, et un reçu consultable est associé.

Scénario nominal

1. Si la station est déjà choisie, l'assistant commence directement au choix du connecteur ; sinon le client sélectionne d'abord une station.
2. Le client choisit un connecteur libre et consulte son type, sa puissance et le tarif effectif.
3. Il branche physiquement le câble. L'étape progresse uniquement lorsque le message **OCPP StatusNotification** indique la préparation ; aucun bouton ne confirme manuellement le branchement.
4. Le service de paiement préautorise un plafond simulé et conserve une clé d'idempotence.
5. Le backend crée une tentative et envoie **RemoteStartTransaction** avec un identifiant OCPP virtuel à courte portée.
6. La session métier n'est créée qu'après le **StartTransaction** de la borne.
7. Les **MeterValues** mettent à jour énergie, puissance, durée et coût provisoire affichés dans la vue live.
8. Le client arrête la recharge ou une limite d'énergie, de montant ou de durée demande l'arrêt automatique.
9. **RemoteStopTransaction** est envoyé ; la borne retourne **StopTransaction** avec le compteur final.
10. Le montant définitif est calculé à partir du tarif figé, capturé auprès du fournisseur et présenté dans le reçu.

Alternatives et exceptions

- si le câble n'est pas détecté, l'assistant reste en attente et permet l'annulation sans session ;
- un refus de préautorisation empêche l'envoi du démarrage ;
- un timeout ou refus OCPP annule la tentative et libère la préautorisation ;
- une coupure pendant la recharge marque la session interrompue ; un événement final tardif peut la réconcilier une seule fois ;
- un webhook dupliqué ne provoque ni double capture ni double revenu.

2.3.8 UC-06 – Traiter une alerte et une intervention

Objectif	Transformer une anomalie détectée en action terrain traçable jusqu'à la résolution.
Acteurs	Opérateur et Technicien ; service email pour les événements critiques ou assignations.
Préconditions	L'alerte appartient à l'organisation et l'opérateur dispose du droit de gestion.
Déclencheur	Création automatique d'une alerte ou signalement autorisé.
Postconditions	L'alerte est résolue ou documentée pour suivi ; le rapport et les preuves restent accessibles aux acteurs autorisés.

Scénario nominal

1. Le moteur crée une alerte avec source, gravité, station, connecteur, échéance SLA et clé de déduplication.
2. L'opérateur l'acquiesce, examine sa chronologie et choisit un technicien actif de la même organisation.
3. Il crée l'intervention et la priorité est transmise au technicien par notification.
4. Le technicien accepte puis démarre l'intervention.
5. Il ajoute diagnostic, notes, actions, pièces, contrôles et photos avant/après.
6. Il termine le rapport avec un résultat : résolu, suivi requis ou non résolu.
7. L'opérateur vérifie le résultat et clôture l'alerte lorsque la condition technique a disparu.

Alternatives et exceptions

- une alerte identique ouverte est mise à jour au lieu d'être dupliquée ;
- une récurrence après résolution rouvre l'alerte et conserve l'historique ;
- seul le technicien assigné peut produire le rapport terrain ;
- un dépassement de SLA déclenche une notification idempotente ;
- une intervention peut être annulée avec motif, mais son historique n'est pas supprimé.

2.3.9 UC-07 – Gérer le cycle commercial d’une organisation

Objectif	Maintenir un lien cohérent entre période d’essai, offre SaaS, limites, accès commercial et facturation de l’organisation.
Acteurs	Super Administrateur et Administrateur ; service de paiement simulé pour les règlements futurs.
Préconditions	L’organisation existe et possède un abonnement commercial ou un essai.
Déclencheur	Échéance, demande de changement ou action du Super Administrateur.
Postconditions	Le statut commercial, les capacités et les factures restent cohérents et audités.

Scénario nominal

1. Le Super Administrateur publie une offre avec prix, cycle, limite de stations, limite d’employés et fonctionnalités.
2. L’organisation issue d’une démonstration reçoit une période d’essai et ses capacités initiales.
3. L’administrateur consulte son offre, l’utilisation réelle, l’échéance et ses factures.
4. Il soumet une demande de changement sans modifier lui-même le contrat.
5. Le Super Administrateur accepte la demande, prolonge l’essai ou affecte une nouvelle offre.
6. Une facture organisationnelle est produite, puis acquittée ou annulée avec un événement d’audit.

Alternatives et exceptions

- une tentative de dépasser le quota d’employés ou de stations est refusée avant création ;
- à l’expiration, l’organisation conserve ses données mais les opérations protégées sont limitées jusqu’à régularisation ;
- une suspension manuelle exige un motif et peut être restaurée ;
- les paiements des clients pour leurs recharges restent séparés de la facture SaaS de l’organisation.

2.3.10 UC-08 – Générer et échanger un rapport

Objectif	Transformer les données du périmètre en analyse adaptée au rôle, export ou rapport interne traçable.
Acteurs	Super Administrateur, Administrateur, Opérateur ou Technicien.
Préconditions	L'utilisateur possède les permissions de consultation, d'export ou d'échange correspondantes.
Déclencheur	Sélection d'une période ou création d'un brouillon.
Postconditions	L'export est téléchargé ou le rapport est envoyé à un destinataire autorisé avec son historique.

Scénario nominal

1. L'utilisateur choisit une période et consulte les indicateurs propres à son rôle : plateforme, organisation, opérations ou terrain.
2. Les agrégations appliquent le même périmètre que les détails et exposent leur formule.
3. Il exporte la vue en PDF, JSON ou CSV, ou crée un rapport interne.
4. Pour un rapport interne, il choisit catégorie, priorité, destinataire, description et pièces jointes.
5. Le brouillon est envoyé ; le destinataire le consulte, le marque comme lu puis peut l'archiver.
6. L'auteur et le destinataire peuvent prévisualiser le document généré selon leurs permissions.

Alternatives et exceptions

- une période sans données produit des zéros et listes vides explicites ;
- un destinataire d'une autre organisation ou non autorisé n'est pas proposé ;
- une pièce interdite ou trop volumineuse est refusée avant envoi ;
- un export volumineux est traité en file puis notifié lorsqu'il est prêt ;
- le document PDF reprend la période, les filtres, l'émetteur et les indicateurs sans présenter de données étrangères au rôle.

Conclusion

La spécification distingue désormais clairement les responsabilités humaines, les systèmes externes et les composants internes. Le backlog de 42 user stories couvre l'entrée dans la plateforme, la supervision OCPP, l'exploitation terrain, la recharge, le commerce et le pilotage, tout en conservant le firmware comme évolution à haut risque. Les exigences non fonctionnelles rendent les objectifs de sécurité, performance, isolation et traçabilité contrôlables. Le chapitre suivant traduira ces besoins en architecture, composants, contrats et modèles de données.

Conception

Introduction

Le chapitre précédent a défini ce que la plateforme doit accomplir. Le présent chapitre décrit comment ces besoins sont traduits en composants, contrats, processus et données. Les choix présentés ne constituent pas une architecture théorique : ils sont alignés sur les dépendances, les services, les routes et les modèles présents dans le dépôt au moment de la rédaction.

La conception poursuit quatre objectifs : séparer les responsabilités métier des adaptateurs techniques, conserver une source de vérité unique, isoler strictement les organisations et rendre les intégrations remplaçables. Cette démarche conduit à un monolithe Laravel modulaire complété par une application React, une passerelle OCPP spécialisée et des services d'infrastructure indépendants.

3.1 Conception générale

3.1.1 Style architectural retenu

ChargeTrackr adopte une architecture **client-serveur** avec une séparation entre la présentation, le cœur métier, la persistance et les intégrations. Le cœur est un **monolithe modulaire Laravel** : les domaines fonctionnels partagent une même application et une même base PostgreSQL, mais leurs règles sont organisées dans des services, politiques, requêtes de validation et ressources dédiés. Cette solution est adaptée à un projet réalisé par un seul stagiaire, car elle évite le coût opérationnel de microservices tout en conservant des frontières explicites.

Une exception volontaire concerne OCPP. Les connexions WebSocket avec les bornes sont longues, concurrentes et orientées protocole. Elles sont donc prises en charge par une **passerelle Python indépendante**. Cette passerelle ne contient aucune règle métier et

n'accède jamais directement à PostgreSQL. Elle traduit OCPP 1.6 JSON en événements signés transmis à Laravel, qui reste l'unique propriétaire des données et des décisions métier.

Principes structurants

- PostgreSQL est la source de vérité de l'application ;
- Laravel est le seul composant autorisé à modifier les données métier ;
- la passerelle OCPP transporte et normalise le protocole sans calculer la disponibilité ;
- le frontend ne décide jamais des autorisations ni des états définitifs ;
- les fournisseurs externes sont placés derrière des contrats ou adaptateurs ;
- les traitements lents ou récurrents sont confiés au worker et au scheduler.

3.1.2 Architecture technique globale

La figure 3.1 montre les composants exécutables et les protocoles qui les relient. Les services en pointillé sont des intégrations externes ou des simulateurs ; ils peuvent être remplacés sans modifier les règles du domaine.

45

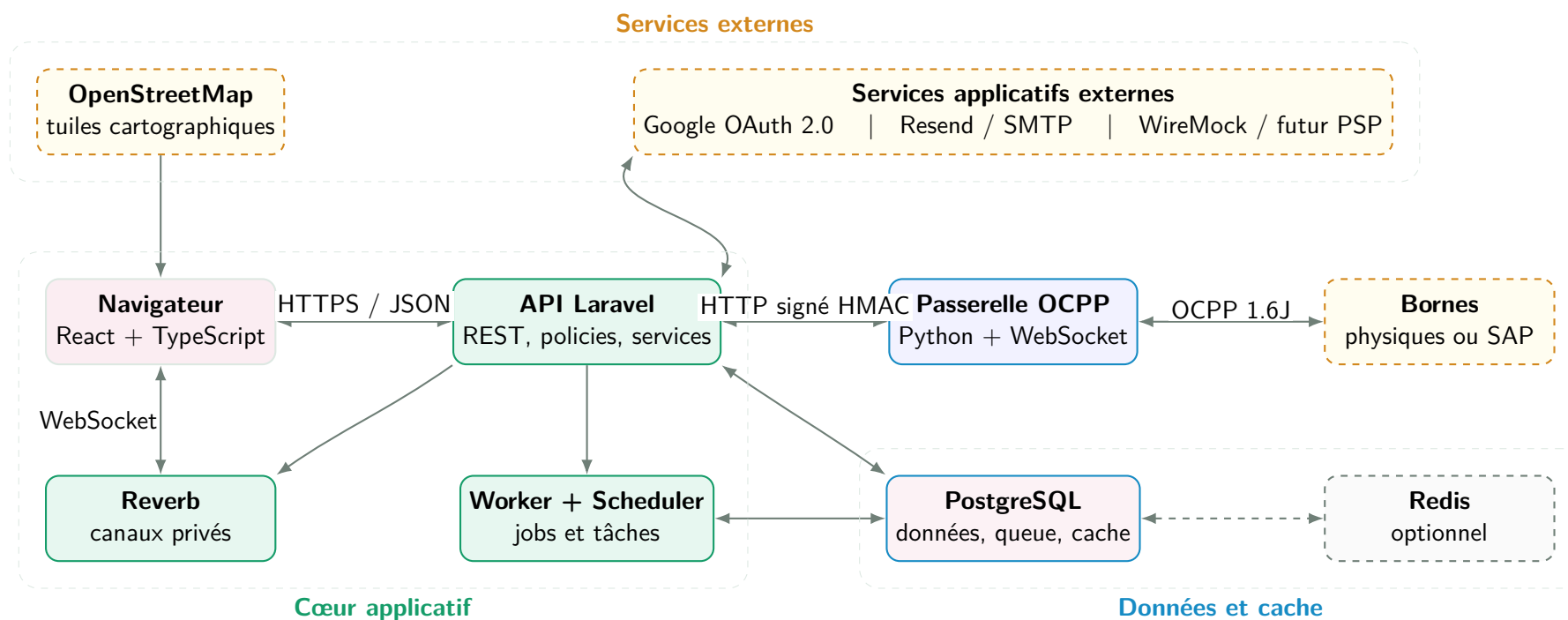


Figure 3.1 – Architecture technique globale de ChargeTrackr

3.1.2.1 Application web React

L'interface est une application monopage écrite en React et TypeScript [16]. Vite assure le serveur de développement et la compilation. Ant Design fournit les contrôles accessibles et cohérents ; TanStack Query gère l'état serveur et son invalidation ; Axios centralise les appels HTTP. React Router applique les routes publiques, protégées, orientées rôle ou permission. Leaflet et OpenStreetMap affichent les bornes et permettent de sélectionner une position [13, 14]. Recharts produit les graphiques et Laravel Echo consomme les événements Verber.

Les appels API sont sortis des composants visuels et regroupés par fonctionnalité dans `src/features`. Cette organisation évite de dupliquer l'authentification, la sérialisation et la gestion d'erreur. L'état serveur reste dans TanStack Query ; les états temporaires d'interface, comme l'étape d'un assistant ou la géolocalisation du navigateur, restent locaux au frontend.

3.1.2.2 API et cœur métier Laravel

Laravel 13 expose l'API REST, valide les entrées, applique les autorisations et orchestre les cas d'utilisation [17]. La chaîne de traitement d'une requête protégée est la suivante :

1. le middleware Sanctum identifie la session et contrôle le cookie CSRF ;
2. le middleware organisationnel refuse les comptes inactifs ou incohérents ;
3. la *Form Request* valide et normalise les données entrantes ;
4. la policy vérifie l'action sur la ressource et son organisation ;
5. le contrôleur appelle un service métier ;
6. le service exécute les invariants, transactions et événements ;
7. une API Resource limite et structure la réponse renvoyée.

Les services couvrent notamment le commissioning des stations, la projection de disponibilité, les transactions OCPP, les tentatives de recharge, le paiement, la maintenance, les notifications, les abonnements et les exports. Les contrôleurs restent ainsi des adaptateurs HTTP plutôt que des conteneurs de logique métier.

3.1.2.3 Persistance, asynchronisme et temps réel

PostgreSQL 16 stocke les comptes, actifs, événements techniques, projections, transactions et journaux [18]. Les opérations atomiques, telles que la création d'une station avec ses

connecteurs ou la réception d'un `StartTransaction`, sont protégées par des transactions et, lorsque nécessaire, des verrous `FOR UPDATE`.

Dans la configuration de référence actuelle, le cache et la file d'attente utilisent également PostgreSQL. Redis 7 est fourni dans le fichier d'infrastructure pour un passage ultérieur à un cache ou une queue dédiés, mais il n'est pas indispensable au fonctionnement local. Cette précision évite de présenter Redis comme une dépendance active alors que `CACHE_STORE=database` et `QUEUE_CONNECTION=database`.

Le worker traite les emails, notifications et captures de paiement. Le scheduler recalcule la disponibilité toutes les 30 secondes, expire les commandes OCPP, génère les maintenances récurrentes, vérifie les SLA et synchronise les abonnements. Reverb diffuse ensuite les changements sur des canaux privés liés à un utilisateur, une organisation ou une session.

3.1.2.4 Passerelle OCPP

La passerelle est réalisée en Python 3.11 avec les bibliothèques `ocpp`, `websockets` et `httpx`. Elle accepte les connexions `ocpp1.6`, authentifie l'identité de la borne auprès de Laravel, publie chaque message normalisé et interroge l'API interne pour les commandes distantes. Elle supporte notamment `BootNotification`, `Heartbeat`, `StatusNotification`, `Authorize`, `StartTransaction`, `MeterValues` et `StopTransaction`, ainsi que les commandes `RemoteStartTransaction`, `RemoteStopTransaction`, `Reset`, `UnlockConnector` et `ChangeAvailability`.

Le protocole OCPP 1.6 JSON est retenu car il correspond au simulateur et au périmètre du stage [3, 12]. OCPP 2.0.1 reste une évolution distincte, car son modèle de sécurité et ses cas d'utilisation nécessitent une validation supplémentaire.

3.1.2.5 Services externes et adaptateurs

Table 3.1 – Intégrations techniques et responsabilités

Intégration	Technologie		Responsabilité	Frontière
Identité sociale	Google	OAuth 2.0	Vérifier l'identité Google et l'email	Les rôles et organisations restent locaux.
Email transactionnel	Resend	SMTP	ou Vérification, invitations, récupération, alertes	Les secrets et l'envoi restent côté serveur.
Cartographie	Leaflet	/ OpenStreetMap	Tuiles, marqueurs, position et itinéraire	La position courante du client n'est pas persistée.
Paieement	Adaptateur	de paiement	Autoriser, capturer, libérer ou débiter	Le fournisseur ne modifie jamais directement une session.
Simulateur de paiement	de	WireMock 3	Succès, refus, délai, erreur et webhook	Aucun moyen de paiement réel n'est utilisé.
Simulateur borne	de	SAP e-mobility simulator	Connexion et messages OCPP réels	Il ne lit ni l'API métier ni la base.

3.1.3 Architecture de déploiement

Le développement local associe des processus Windows et des conteneurs Docker. Les ports ne décrivent pas une obligation de production ; ils rendent les interactions reproductibles sur la machine de développement.

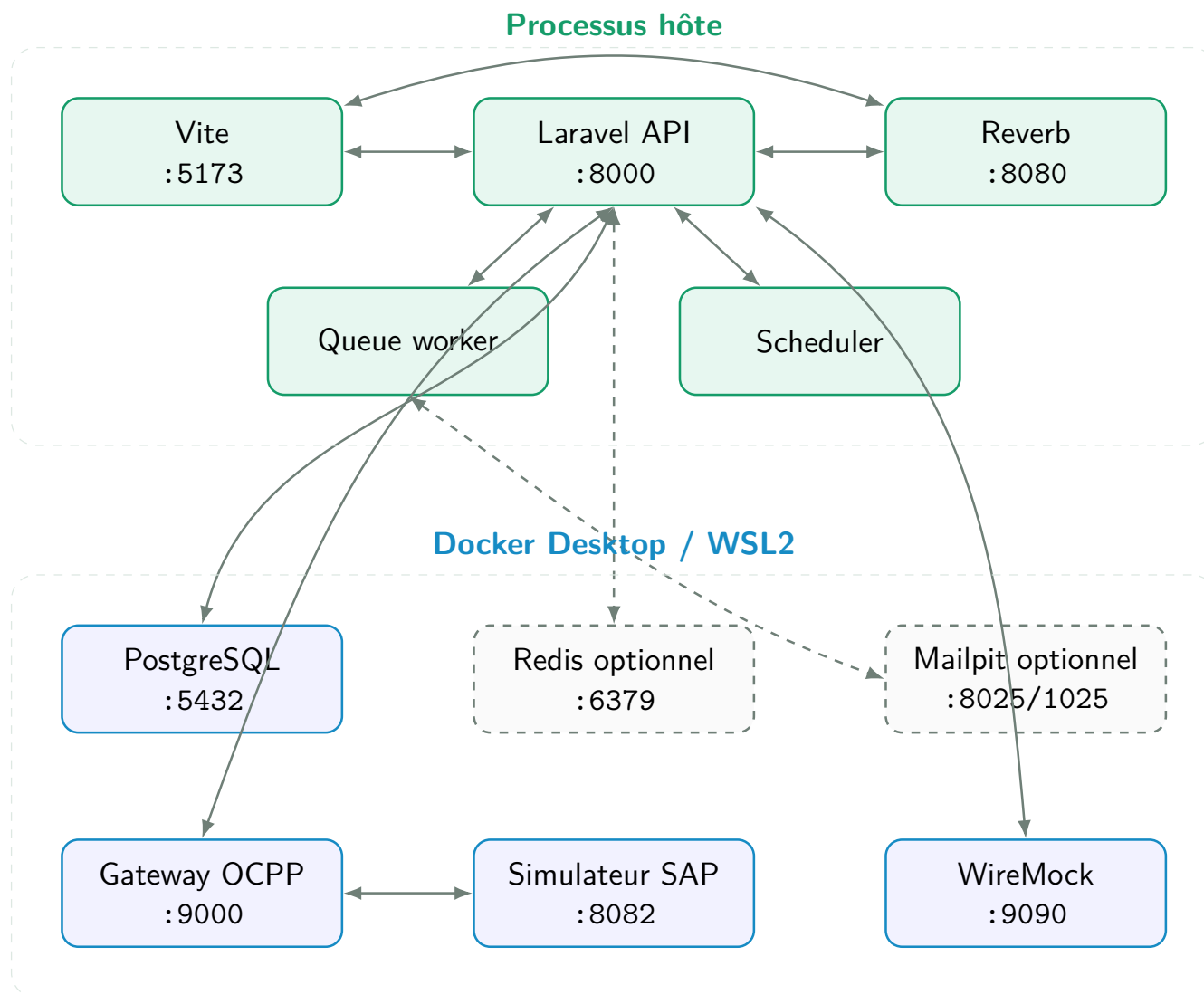


Figure 3.2 – Architecture de déploiement de l'environnement local

Table 3.2 – Processus nécessaires dans l’environnement de développement

Processus	Port	Exécution	Rôle
Frontend Vite	5173	Hôte	Interface React et rechargement à chaud.
API Laravel	8000	Hôte	API REST, OAuth, webhooks et API interne OCPP.
Queue worker	—	Hôte	Emails, notifications et captures différées.
Scheduler	—	Hôte	Disponibilité, SLA, maintenance et synchronisations.
Reverb	8080	Hôte	WebSocket temps réel de l’application.
PostgreSQL	5432	Local ou Docker	Source de vérité et queue/cache par défaut.
Redis	6379	Docker, optionnel	Cache ou queue de remplacement.
Passerelle OCPP	9000	Docker	WebSocket OCPP et adaptateur vers Laravel.
Simulateur SAP	8082	Docker	Flotte de bornes et interface de simulation.
WireMock	9090	Docker	API de paiement externe simulée.
Mailpit	8025/1025	Docker, optionnel	Boîte de réception locale si SMTP est utilisé.

En production, Vite est remplacé par des fichiers statiques servis derrière HTTPS, Laravel est exécuté par un serveur PHP supervisé, et tous les flux publics utilisent TLS. Les connexions des bornes passent de `ws://` à `wss://`. Les workers, le scheduler, Reverb et la passerelle OCPP doivent être surveillés et redémarrés automatiquement. Les secrets restent injectés par l’environnement et ne sont jamais incorporés aux images ni au dépôt.

3.1.4 Architecture logicielle

3.1.4.1 Organisation du frontend

Le frontend suit une organisation par fonctionnalités :

- **features/auth** contient la session, les gardes de routes et la configuration des rôles ;
- **features/stations, maps** et **charging** regroupent les parcours de station et de recharge ;
- **features/operations** couvre alertes, interventions et maintenance ;
- **features/commercial, subscriptions** et **tariffs** séparent les deux niveaux de commerce ;
- **features/realtime** encapsule Echo/Reverb ;
- **pages** compose ces modules sans réimplémenter leurs contrats API.

L'interface dérive ses menus et tableaux de bord du rôle retourné par le backend. Cette adaptation améliore l'expérience utilisateur, mais elle n'est pas utilisée comme barrière de sécurité : une route masquée reste protégée côté Laravel.

3.1.4.2 Organisation du backend

Le backend suit les responsabilités de Laravel sans imposer une couche générique de *repository* au-dessus d'Eloquent :

Http/Controllers

adaptation des requêtes et réponses ;

Http/Requests validation, normalisation et préconditions ;

Policies autorisation par action, rôle et organisation ;

Services orchestration transactionnelle et règles métier ;

Models relations, attributs, conversions et invariants locaux ;

Jobs/Events traitements asynchrones et diffusion temps réel ;

Contracts interfaces des intégrations remplaçables.

L'interface **PaymentGateway** illustre cette séparation. Les services de recharge appellent les opérations **authorize**, **capture**, **release** et **charge** sans connaître WireMock. L'injection de dépendances sélectionne l'adaptateur selon **PAYMENT_DRIVER**. Un futur adaptateur ClicToPay, e-DINAR/D17, GPGCheckout, Konnect ou Flouci pourra respecter le même contrat.

3.2 Conception détaillée

3.2.1 Identité, demandes de démonstration et invitations

Trois parcours créent des comptes, chacun avec une règle différente :

1. un visiteur peut créer directement un compte **client**. L'adresse doit être vérifiée avant la connexion locale ;
2. un client peut utiliser Google OAuth. L'identité Google est liée à un compte local, mais ne fournit ni rôle ni organisation ;
3. un administrateur, opérateur ou technicien est toujours créé par invitation. Le destinataire active lui-même son compte et choisit son mot de passe.

La demande de démonstration constitue l'entrée commerciale d'une organisation. Le visiteur renseigne ses coordonnées, la société, la taille du réseau et plusieurs objectifs opérationnels. Une confirmation est envoyée au demandeur sans lien interne. Le super administrateur ouvre ensuite la demande, la rejette ou lance sa revue. Une acceptation provisionne, dans une transaction, l'organisation, son administrateur en état **pending**, l'essai commercial et une invitation temporaire. Le compte ne devient actif qu'après acceptation du lien à usage unique.

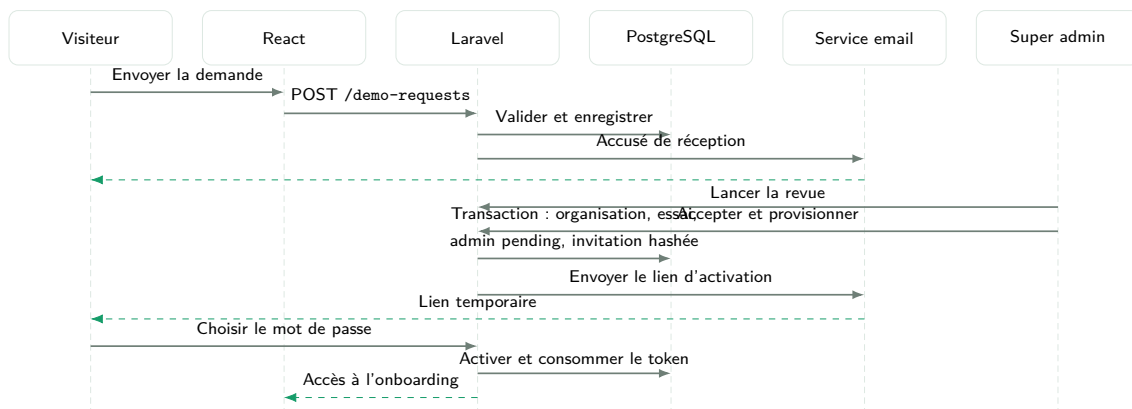


Figure 3.3 – Provisionnement d'une organisation à partir d'une demande de démonstration

Les invitations suivent les invariants suivants :

- le token aléatoire n'est envoyé qu'une fois et seul son condensat SHA-256 est stocké ;
- le lien possède une expiration et devient inutilisable après acceptation, révocation ou remplacement ;
- un rappel renouvelle le token, car sa valeur originale ne peut pas être récupérée ;

- un administrateur ne peut inviter que des opérateurs ou techniciens de son organisation ;
- l'utilisateur invité choisit son secret ; l'administrateur ne connaît jamais son mot de passe.

3.2.2 Mise en service d'une station

Le commissioning rassemble la station, son emplacement, son matériel, ses connecteurs et sa cible OCPP dans un assistant unique. La création est atomique : si une référence, une identité OCPP ou un connecteur est invalide, aucune partie incomplète n'est conservée.

Table 3.3 – Cibles de mise en service OCPP

Cible	Comportement	Usage
Borne physique ou externe	Génération d'un secret Basic Auth individuel de 48 caractères, affiché une seule fois et stocké uniquement sous forme de hash.	Intégration future d'un chargeur réel ou d'un simulateur tiers.
Simulateur SAP local	Création de l'inventaire puis ajout du profil au manifeste local par une commande Artisan dédiée.	Tests reproductibles du stage.
Inventaire seul	Création sans identité ni secret OCPP. L'état reste non disponible jusqu'au provisioning.	Préparation administrative d'un futur actif.

La référence interne et l'identité OCPP peuvent différer, mais leur égalité simplifie l'exploitation. Le libellé A1 d'un connecteur est destiné à l'utilisateur ; `ocpp_connector_id` est l'entier réellement transmis par la borne. Pour le simulateur SAP, ces identifiants doivent être contigus à partir de 1. Après provisionnement, la borne se connecte à `/ocpp/<identité>` avec le sous-protocole `ocpp1.6`.

3.2.3 Ingestion OCPP et disponibilité calculée

3.2.3.1 Séparation entre preuve brute et projection métier

Un statut reçu de la borne n'est pas directement présenté comme vérité métier. L'événement complet est d'abord enregistré dans `ocpp_events`. Les champs `ocpp_*` de la station et du connecteur forment ensuite une projection technique, tandis que les champs `status`, `availability_reason` et `availability_source` constituent la projection métier.

Cette séparation apporte trois garanties :

- un événement tardif ne remplace pas une preuve plus récente ;
- les règles peuvent évoluer sans supprimer l'historique technique ;
- chaque changement visible peut être expliqué par une transition et son événement source.

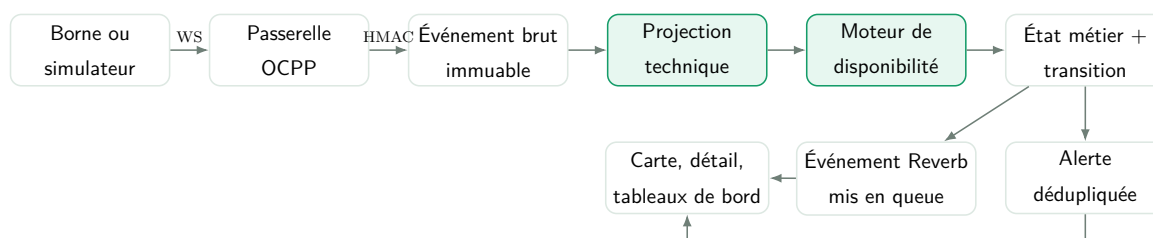


Figure 3.4 – Chaîne de traitement d'un événement OCPP

3.2.3.2 Règles de disponibilité

La résolution applique les règles par priorité. Une décision manuelle ou une maintenance planifiée domine toujours la télémétrie. Ensuite, une fermeture WebSocket explicite ou 90 secondes sans message produit l'état `offline`. Le délai représente trois périodes de heartbeat de 30 secondes. Enfin, les états des connecteurs déterminent la capacité réelle de la station.

Table 3.4 – Matrice de décision de disponibilité

Prio.	Preuve ou règle	État station	Conséquence
1	Désactivation manuelle	<code>unavailable</code>	Recharge interdite jusqu'à levée de l'override.

Prio.	Preuve ou règle	État station	Conséquence
2	Maintenance active	<code>maintenance</code>	Indisponibilité planifiée et commande <code>ChangeAvailability</code> .
3	Déconnexion ou silence supérieur à 90 secondes	<code>offline</code>	Connecteurs hors ligne et alerte de communication.
4	Au moins un connecteur <code>Available</code>	<code>available</code>	Démarrage distant autorisé si l'organisation est active.
5	Aucun libre, au moins un en transaction	<code>charging</code>	Station vivante mais sans place libre correspondante.
6	Aucun libre, au moins un réservé	<code>reserved</code>	Réservation visible ; recharge publique non immédiate.
7	Aucun utilisable, au moins un <code>Faulted</code>	<code>faulted</code>	Alerte de défaut dédoublée.
8	Connectée sans preuve exploitable	<code>unavailable</code>	Attente d'un <code>StatusNotification</code> valide.

Chaque variation de l'état, de la raison ou de la source crée une `AvailabilityTransition`. Les alertes automatiques utilisent une clé de déduplication stable : la même panne ne crée pas une série d'alertes identiques, mais une récurrence après résolution rouvre le suivi. L'événement `StationAvailabilityChanged` invalide les vues React concernées via `Reverb`.

3.2.4 Parcours de recharge et transaction OCPP

3.2.4.1 Assistant côté client

Le parcours client suit cinq étapes afin de correspondre aux actions physiques :

1. choisir une station publique disponible ou ouvrir directement une station ;

2. sélectionner un connecteur compatible et libre ;
3. brancher le câble ; l'interface attend réellement le statut OCPP **Preparing** et n'utilise pas une confirmation manuelle ;
4. choisir le moyen de paiement et, facultativement, une limite d'énergie, de montant ou de durée ;
5. autoriser le paiement puis demander le démarrage distant.

Le rayon *Near me* est une préférence de compte. La géolocalisation n'est demandée qu'après une action explicite et reste dans le navigateur. Un QR de connecteur contient un lien profond vers la station et le connecteur ; il ne contient ni idTag, ni secret OCPP, ni jeton de paiement.

3.2.4.2 Tentative, session et transaction

Trois objets distincts évitent de facturer une énergie qui n'a pas été délivrée :

ChargingAttempt

intention du client, préautorisation et commande de démarrage ;

OcppTransaction

preuve technique issue de la borne ;

ChargingSession

session métier créée seulement après un **StartTransaction** accepté.

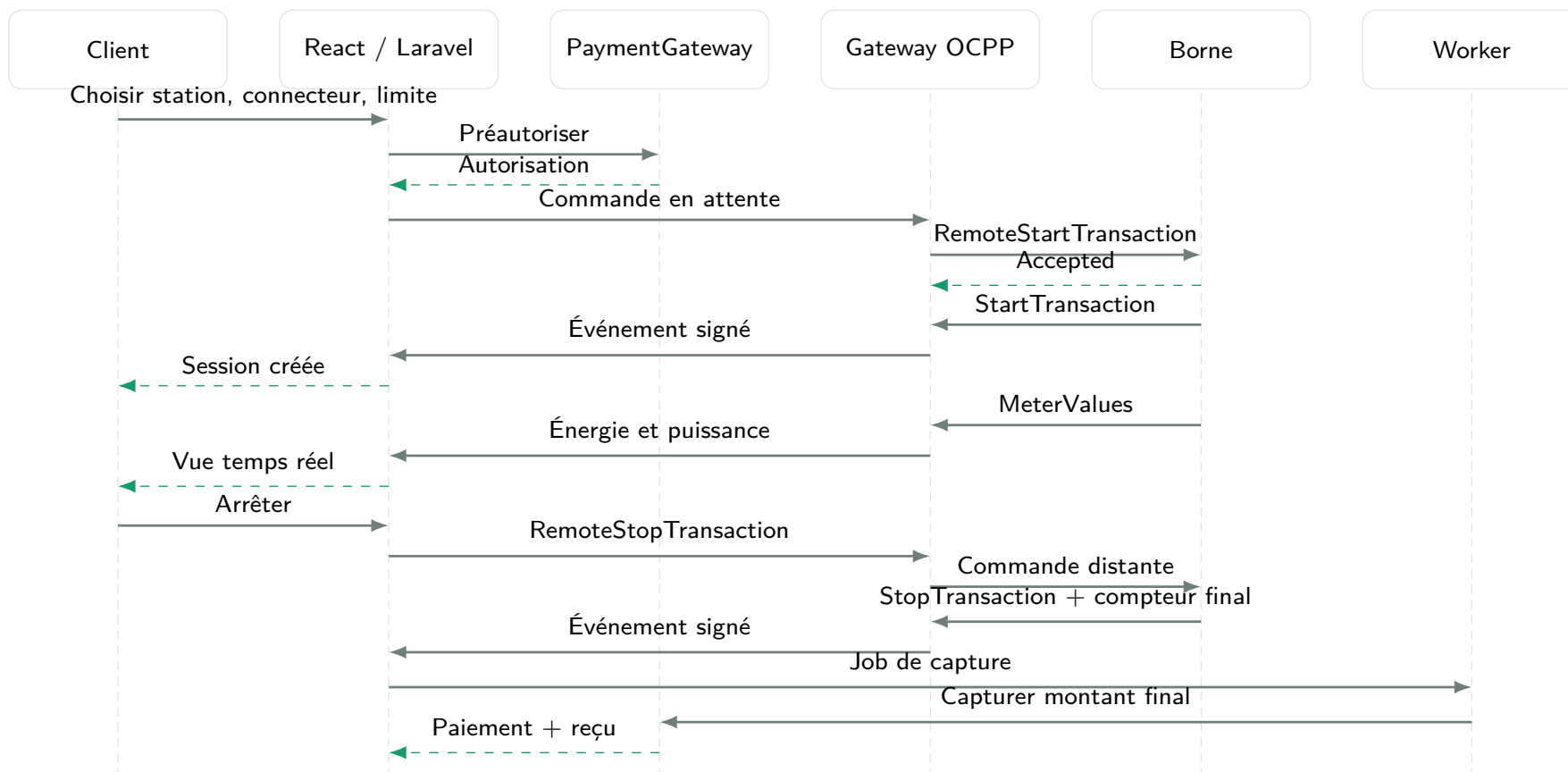


Figure 3.5 – Séquence de recharge distante et paiement simulé

Une préautorisation réussie ne prouve donc pas que la recharge a commencé. Laravel crée une commande **RemoteStartTransaction** chiffrée au repos et limitée dans le temps. La passerelle la réclame, l'envoie à la connexion active, puis retourne le résultat. Seul le **StartTransaction** ultérieur crée la session facturable. Les **MeterValues** actualisent l'énergie, la puissance, l'état de charge et l'estimation de coût. Une limite atteinte produit une commande **RemoteStopTransaction**.

L'arrêt manuel du client suit le même mécanisme distant. Le **StopTransaction** final fournit le compteur et la raison de fin. Si la borne se déconnecte avant ce message, la transaction passe en **awaiting_reconciliation** et la session est interrompue sans inventer de compteur final. Un message tardif peut la réconcilier.

3.2.5 Paiement et reçu

Le paiement applique le patron **Adapter**. Le domaine dépend de **PaymentGateway**, pas d'un fournisseur. Le contrat expose quatre opérations : préautoriser, capturer, libérer et débiter. Deux implémentations existent :

- **SimulatedPaymentAdapter**, rapide et déterministe pour les tests ;
- **WireMockPaymentAdapter**, qui appelle une API HTTP externe simulée.

WireMock reproduit quatre résultats : succès, refus métier, délai dépassé et erreur fournisseur. Les callbacks portent une signature HMAC-SHA256. Laravel vérifie la signature, enregistre l'identifiant fournisseur une seule fois et protège chaque opération par une clé d'idempotence. Une répétition du webhook ne peut donc ni capturer deux fois le paiement ni doubler le chiffre d'affaires.

La préautorisation par défaut est de 30 TND et expire après 15 minutes. Elle est libérée si la station refuse le démarrage ou si la tentative expire. Après **StopTransaction**, le worker capture uniquement le montant final calculé à partir du tarif figé au démarrage. Le reçu PDF est généré côté serveur à partir de la session et du paiement persistés. Il peut être prévisualisé ou téléchargé sans exposer les données d'un autre client.

3.2.6 Alertes, interventions et maintenance

Ces trois notions appartiennent au même cycle opérationnel mais répondent à des questions différentes :

Alerte	Quel problème a été détecté, avec quelle gravité et quel SLA ?
---------------	--

Intervention Qui traite concrètement ce problème, quand et avec quelles preuves ?

Maintenance Quelle action préventive ou corrective doit être planifiée, éventuellement de façon récurrente ?

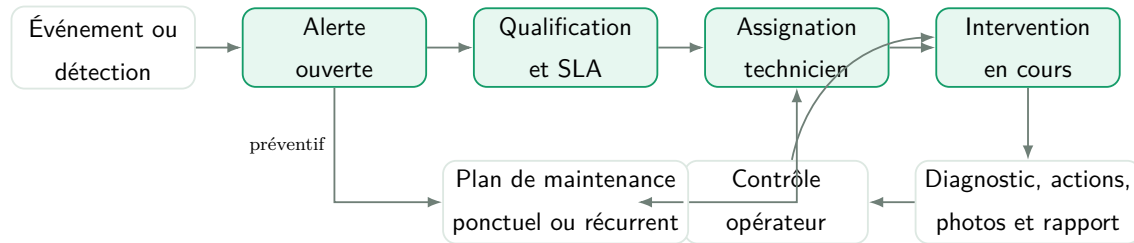


Figure 3.6 – Cycle opérationnel des alertes, interventions et maintenances

L’opérateur qualifie l’alerte, assigne un technicien et peut créer une intervention. Le technicien accepte le travail, ajoute notes, diagnostic, photos avant/après et rapport, puis indique sa résolution. L’opérateur contrôle les preuves et clôture le suivi. Les changements sont conservés dans des journaux d’événements, et les notifications SLA sont dédoublées pour ne pas produire de rappels multiples.

Une maintenance peut exister sans alerte, par exemple un contrôle trimestriel. Le scheduler génère ses occurrences et peut les transformer en interventions assignées. À l’inverse, une intervention corrective peut être créée directement depuis une alerte. Les documents et photos sont conservés sur un disque privé et servis uniquement après vérification de policy.

3.2.7 Tarification, plans et gestion commerciale

La conception distingue deux marchés qui ne doivent pas être mélangés :

- le **tarif de recharge** appartient à l’organisation exploitante et calcule le coût d’une session selon le kWh, les frais de session, l’inactivité et un minimum ;
- le **plan client** est un abonnement du conducteur à une organisation et peut fournir une remise. Un client peut posséder plusieurs abonnements auprès d’organisations différentes ;
- le **plan SaaS** est le contrat entre la plateforme et une organisation. Il fixe notamment les limites d’employés et de stations.

Le tarif effectif est résolu du plus spécifique au plus général : affectation au connecteur, à la station, puis tarif par défaut de l’organisation. Lors du démarrage, la session conserve

un instantané du tarif et du plan client. Une modification ultérieure ne change donc pas rétroactivement une recharge en cours.

La demande de démonstration crée un essai commercial. À l'échéance, le cycle de vie peut entrer en période de grâce, être activé sur une offre ou suspendre les opérations dépassant les droits. Le super administrateur gère le catalogue SaaS, les essais et les factures ; l'administrateur d'organisation ne consulte que son contrat, son usage et ses demandes de changement.

3.2.8 Rapports, documents et exports

Les analyses ne sont pas un écran générique dont seules les valeurs changent. Le super administrateur suit l'adoption et le portefeuille d'organisations ; l'administrateur suit les revenus et les capacités ; l'opérateur suit la disponibilité, les incidents et l'énergie ; le technicien suit ses SLA et résultats terrain. Les contrôleurs de reporting exposent donc des contrats distincts `platform`, `organization`, `operations` et `field`.

Les exports CSV et JSON utilisent les mêmes filtres et le même périmètre que la vue. Les PDF sont générés côté serveur afin de garantir une identité visuelle stable et une source vérifiée. Les rapports internes possèdent un émetteur, un destinataire, un statut, des pièces jointes privées et une version PDF. Ils permettent les passations entre employés et l'envoi d'un bilan à l'administrateur sans utiliser un canal externe non audité.

3.3 Modèle de données et diagrammes de classes

3.3.1 Principes de modélisation

Le schéma relationnel est organisé autour de l'organisation, de la station et de la session. La majorité des enregistrements opérationnels possèdent un `organization_id`. Cette redondance contrôlée permet de filtrer rapidement les requêtes, d'appliquer les politiques et de conserver le périmètre d'une preuve même lorsque l'association technique est chargée séparément.

Le modèle distingue les événements immuables des projections modifiables. Les journaux OCPP, d'alertes, d'interventions, d'abonnements et d'audit expliquent l'historique ; `Station`, `Connector`, `Alert` ou `ChargingSession` portent l'état courant nécessaire à l'interface. Ce choix ne constitue pas un *event sourcing* complet : les événements les plus critiques sont conservés, mais les agrégats restent persistés directement.

Les montants sont stockés en **millimes entiers** afin d'éviter les erreurs d'arrondi des nombres flottants. Les dates techniques sont normalisées en UTC ; le fuseau du compte sert uniquement à l'affichage. Les suppressions logiques sont utilisées pour les actifs et tarifs qui doivent rester référencés par l'historique.

3.3.2 Identité, actifs et recharge

La figure 3.7 présente le noyau qui relie l'organisation, les utilisateurs, les bornes, OCPP, la session et le paiement. Pour alléger la figure, toutes les associations organisationnelles secondaires ne sont pas dessinées ; leur présence est signalée par l'attribut `organization_id`.

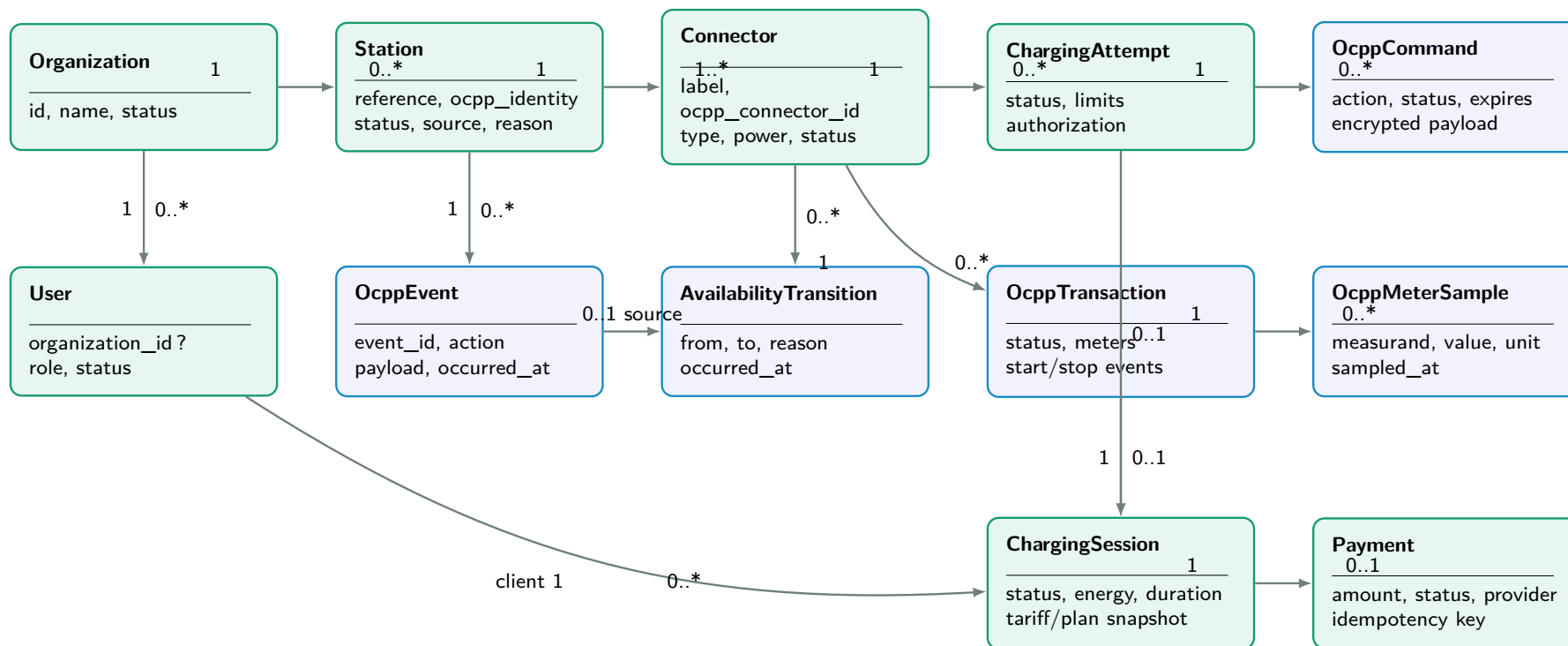


Figure 3.7 – Diagramme de classes conceptuel du noyau station-recharge

3.3.2.1 Organisation et utilisateur

Une organisation possède ses employés, stations, tarifs, plans, sessions, paiements et opérations. Un utilisateur porte exactement un rôle principal. `admin`, `operator` et `technician` appartiennent à une seule organisation active. `super_admin` et `client` sont globaux et conservent `organization_id` à `null`. Le client peut néanmoins utiliser les bornes de plusieurs organisations : la relation est portée par ses sessions et abonnements, pas par son compte.

3.3.2.2 Station et connecteur

Une station appartient à une organisation et possède un ou plusieurs connecteurs. Elle contient à la fois les données d'inventaire, la dernière preuve OCPP et la projection de disponibilité. Le connecteur associe un libellé lisible, un identifiant OCPP, un type physique, un courant, une puissance et deux statuts : brut et métier. L'unicité de l'identifiant OCPP est imposée dans le périmètre de la station.

3.3.2.3 Transaction et session

`OcppTransaction` représente la transaction du protocole, même si elle est refusée. Elle référence ses événements de début et de fin et contient plusieurs `OcppMeterSample`. `ChargingSession` n'existe que pour une transaction acceptée et conserve le client, l'énergie, le coût, les limites et les instantanés de tarification. Une session possède au plus un paiement final.

3.3.3 Exploitation et commerce

La figure 3.8 regroupe les agrégats qui complètent le noyau : traitement des incidents, documents, tarification, abonnements clients et contrat commercial de l'organisation.

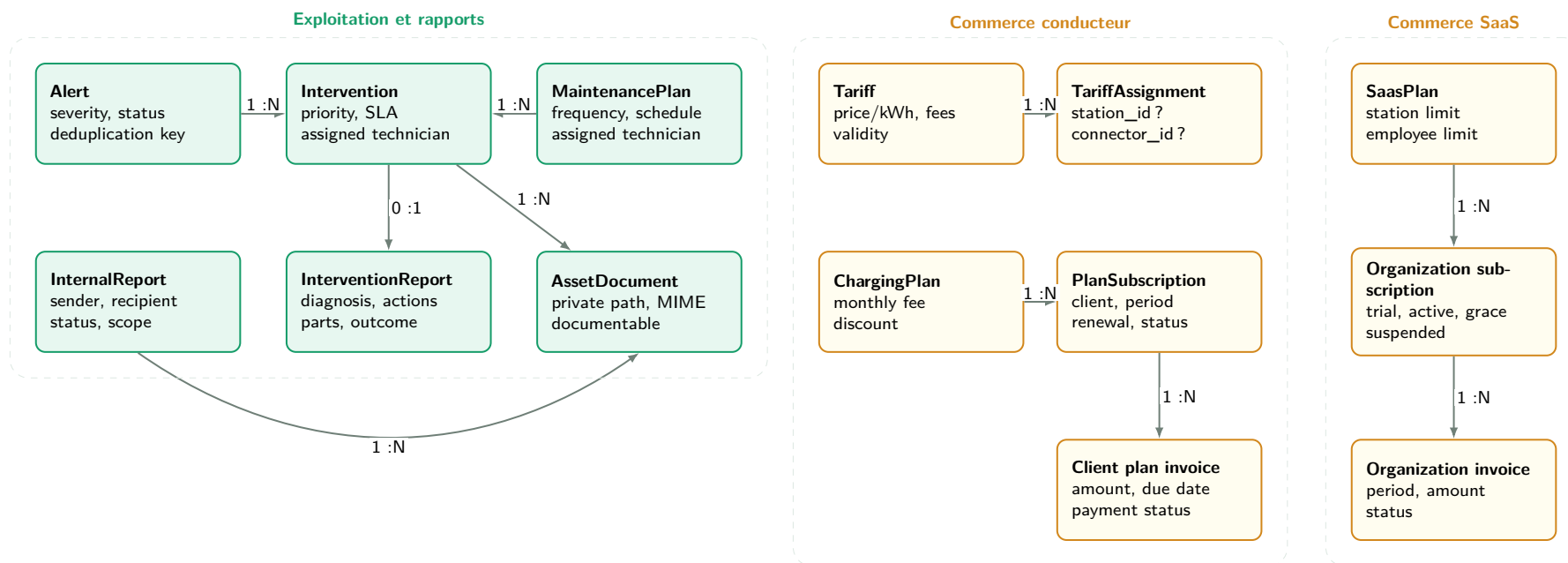


Figure 3.8 – Diagramme de classes conceptuel des domaines d’exploitation et commerciaux

3.3.3.1 Agrégat d'exploitation

Une alerte appartient à une station et, éventuellement, à un connecteur. Elle possède un historique et peut produire plusieurs interventions dans le temps. Une intervention peut également provenir d'un plan de maintenance. L'assignation pointe vers un technicien de la même organisation. Les photos, le rapport structuré et les documents polymorphes constituent les preuves terrain.

3.3.3.2 Agrégat de tarification client

Un tarif appartient à une organisation et possède plusieurs affectations. Une affectation cible soit une station, soit un connecteur. Un plan de recharge décrit une offre client ; `PlanSubscription` relie un client global à ce plan et à l'organisation. Les factures d'abonnement client sont distinctes des paiements d'une session de recharge.

3.3.3.3 Agrégat commercial SaaS

`SaaSPlan` définit les droits vendus aux organisations. `OrganizationSubscription` matérialise l'essai, l'offre active, la période de grâce ou la suspension. Ses événements assurent la traçabilité du cycle de vie, et `OrganizationInvoice` représente la facturation entre la plateforme et l'organisation. Cette modélisation évite de mélanger le conducteur qui paye une recharge et l'organisation qui paye l'accès au logiciel.

3.3.4 Contraintes et invariants principaux

Table 3.5 – Invariants structurants du modèle

Domaine	Invariant
Identité	Un utilisateur actif possède exactement un rôle ; un rôle organisationnel exige une organisation active, alors qu'un rôle global interdit cette association.
Multi-tenant	Toute lecture ou écriture opérationnelle est limitée à l'organisation de l'utilisateur, sauf permission explicite du super administrateur.

Domaine	Invariant
Commissioning	La station et ses connecteurs sont créés dans une même transaction ; la référence et l'identité OCPP sont uniques.
Disponibilité	Le statut métier est dérivé des overrides et preuves OCPP récentes ; il n'est pas saisi directement pour une station provisionnée.
Recharge	Un client et un connecteur ne peuvent participer qu'à une tentative ou session active à la fois.
OCPP	Un <code>StartTransaction</code> refusé reste auditable mais ne crée pas de session facturable.
Mesure	L'énergie ne diminue jamais ; le compteur effectif est le maximum entre le départ, la dernière mesure et le compteur reçu.
Paielement	Chaque opération possède une clé d'idempotence ; un événement fournisseur est traité une seule fois.
Tarification	Le tarif et la remise sont figés au début de la session.
Intervention	Le technicien assigné, la station et l'intervention appartiennent à la même organisation.
Documents	Le contenu privé n'est jamais exposé par une URL publique permanente.

3.4 Sécurité et isolation multi-tenant

3.4.1 Authentification de l'application web

L'application React est un client propriétaire de la même plateforme. Elle utilise donc Laravel Sanctum en mode **session stateful** plutôt qu'un JWT conservé dans le stockage du navigateur. Avant une connexion locale, le frontend récupère le cookie CSRF et envoie les requêtes avec les cookies activés. Le cookie de session est `HttpOnly`, ce qui réduit l'exposition du secret de session au code JavaScript.

Google OAuth est orchestré par le backend. Le secret client Google n'est jamais fourni au frontend. Après le callback, Laravel exige un email vérifié, lie ou crée le compte local

puis établit la même session Sanctum. Un compte Google ne reçoit aucun rôle privilégié du fournisseur : les rôles et l'organisation sont attribués uniquement par la plateforme. Dans le périmètre actuel, Microsoft OAuth est différé.

Les mots de passe locaux sont hashés par le mécanisme Laravel. Les formulaires de récupération répondent de manière générique afin de limiter l'énumération des comptes. Les comptes exclusivement Google n'affichent pas une action de changement de mot de passe tant qu'une authentification locale n'a pas été activée.

3.4.2 Autorisation et périmètre organisationnel

L'autorisation repose sur trois niveaux complémentaires :

1. les permissions Spatie décrivent les capacités d'un rôle ;
2. les politiques contrôlent l'action sur la ressource concrète ;
3. `canAccessOrganization` et les filtres de requête garantissent le périmètre de données.

Table 3.6 – Périmètre d'accès des rôles

Rôle	Association	Périmètre
Super administrateur	Global, sans organisation	Organisations, intégrations, audit et commerce SaaS selon ses permissions.
Administrateur	Une organisation	Employés, clients, actifs, tarifs, commerce et rapports de son organisation.
Opérateur	Une organisation	Stations, connecteurs, sessions, alertes, maintenances et interventions de son organisation.
Technicien	Une organisation	Consultation des actifs et traitement des alertes, interventions ou maintenances autorisées, sans création de station.
Client	Global, sans organisation	Stations publiques et uniquement ses propres tentatives, sessions, paiements, abonnements et notifications.

Le frontend adapte la navigation, mais l'absence d'un bouton ne constitue jamais une

autorisation. Chaque endpoint sensible appelle une policy. L'administrateur ne peut pas gérer les employés d'une autre organisation, changer son propre rôle, désactiver son propre compte ni supprimer le dernier administrateur actif. Les identifiants présents dans l'URL sont donc revalidés afin de prévenir les accès directs de type IDOR.

3.4.3 Sécurité des frontières OCPP

Deux niveaux protègent la communication des bornes :

- la borne utilise une identité OCPP et un secret Basic Auth. Pour une borne externe, le secret aléatoire est affiché une seule fois et seul son hash est stocké ;
- la passerelle signe ses requêtes internes avec HMAC-SHA256 sur le corps exact, un timestamp Unix et un identifiant UUID.

Laravel rejette les signatures invalides, les timestamps trop anciens et les UUID déjà utilisés. L'identifiant métier de l'événement rend également l'ingestion idempotente : une répétition identique retourne l'enregistrement existant, tandis qu'un même identifiant associé à un contenu différent est refusé. Les payloads de commandes sont chiffrés au repos et expirent. Un seul gateway peut réclamer une commande en attente.

HMAC garantit l'intégrité et l'authenticité du service, mais pas la confidentialité. En production, la borne utilise donc `wss://` et le gateway communique avec Laravel par HTTPS ou au sein d'un réseau privé chiffré. Le secret partagé du simulateur n'est acceptable que dans l'environnement local ; chaque borne physique doit recevoir un secret individuel.

3.4.4 Sécurité du paiement

Le simulateur n'accepte aucune carte réelle. L'API externe reçoit une référence, un montant, une devise, un résultat de simulation et une clé d'idempotence. Le webhook retour est signé et limité en fréquence. `PaymentProviderEvent` conserve l'identifiant du fournisseur, la signature vérifiée et le résultat de réconciliation. Les règles suivantes s'appliquent :

- aucune confirmation du frontend ne marque directement un paiement comme réussi ;
- un webhook invalide n'est pas traité ;
- une répétition valide ne produit pas une seconde opération ;
- un délai ou une indisponibilité technique est distingué d'un refus métier ;

— les secrets, numéros de carte et payloads sensibles ne sont jamais journalisés.

L'intégration d'un prestataire réel nécessitera en plus la conformité aux exigences du fournisseur, TLS public, une politique de remboursement, la réconciliation financière et, selon le mode choisi, l'externalisation complète des données de carte.

3.4.5 Validation, limitation et gestion des erreurs

Les écritures utilisent des Form Requests et retournent une erreur métier contrôlée plutôt qu'une trace SQL ou une exception interne. Des buckets de limitation indépendants empêchent un parcours public de bloquer un autre :

Table 3.7 – Principales limites applicatives

Flux	Limite	Clé
Demande de démonstration	3 par heure	Adresse IP
Inspection d'invitation	10 par minute	Adresse IP
Acceptation d'invitation	5 par minute	IP et hash de l'email
Gestion des invitations employé	10 par minute	Utilisateur authentifié
API interne OCPP	600 par minute	Adresse de la passerelle
Webhook de paiement	120 par minute	Adresse du fournisseur
Changement de mot de passe	5 par minute	Session authentifiée

La déduplication métier complète le throttling. Par exemple, une demande récente ayant la même identité n'est pas recréée, et les alertes automatiques utilisent une clé stable. Cette combinaison traite à la fois les abus volumétriques et les doubles clics légitimes.

3.4.6 Secrets, fichiers et journalisation

Les fichiers `.env` contiennent les clés Laravel, Google, Resend, Reverb, OCPP et paiement. Ils sont ignorés par Git ; seuls les `.env.example` sans valeurs privées sont versionnés. Le frontend ne reçoit que les paramètres publics préfixés `VITE_`. Les secrets de bornes, tokens OAuth, mots de passe, en-têtes `Authorization` et signatures ne doivent pas apparaître dans les logs.

Les photos d'intervention, rapports et documents techniques utilisent le disque Laravel privé. Un contrôleur autorisé lit le fichier et le diffuse après policy ; une URL de stockage publique ne contourne donc pas le périmètre organisationnel. Les types MIME, tailles et extensions sont validés avant enregistrement.

`PlatformAuditLog` conserve les actions sensibles : gestion de comptes et rôles, changement de paramètres, commissioning, rotation de credentials, tarifs, abonnements et états financiers. L'auteur, l'organisation, la cible, l'action et un résumé sont enregistrés sans inclure le secret modifié. Une tâche quotidienne applique la politique de rétention.

3.4.7 Analyse synthétique des risques

Table 3.8 – Risques principaux et mesures de réduction

Risque	Exemple	Mesures
Vol de session	Script lisant un token du navigateur	Cookie HttpOnly, Sanctum stateful, CSRF, rotation et TLS.
IDOR multi-tenant	Accès à <code>/stations/11</code> d'une autre organisation	Policies, scoping organisationnel, tests d'isolation.
Rejeu OCPP	Réémission d'un événement signé	Timestamp, UUID de requête, HMAC et identifiant d'événement idempotent.
Faux paiement	Callback frontend ou webhook forgé	Confirmation serveur, signature HMAC et événements fournisseur uniques.
Double facturation	Répétition d'une capture après timeout	Clés d'idempotence et réconciliation persistée.
Exposition documentaire	URL publique d'une photo technique	Stockage privé, validation et téléchargement soumis à policy.
Secret dans Git	Commit d'un client secret OAuth	<code>.gitignore</code> , exemples vides et injection par environnement.
Déni de service logiciel	Soumission répétée de demandes ou invitations	Rate limiting indépendant, honeypot et déduplication.

3.4.8 Limites de sécurité du prototype

Le socle protège les parcours implémentés, mais ne remplace pas un audit de production. Il reste notamment à automatiser un scan des dépendances, renforcer les en-têtes HTTP, exécuter des tests de charge OCPP, définir la rotation des secrets, déployer une terminaison TLS, superviser les processus et tester les sauvegardes. Une borne physique et un fournisseur de paiement réel devront être homologués séparément. Ces limites sont documentées afin de ne pas confondre un MVP sécurisé par conception avec une certification d'exploitation.

Conclusion

La conception retenue combine la simplicité transactionnelle d'un monolithe modulaire et l'isolation d'une passerelle dédiée aux connexions OCPP longues. Les états métier ne sont jamais assimilés aux messages bruts des bornes : des services de projection reconstruisent la disponibilité, les sessions et les alertes à partir de preuves horodatées. La séparation par contrats rend le paiement, la messagerie et les équipements remplaçables, tandis que les politiques et le *scoping* organisationnel protègent les données. Le chapitre suivant montrera comment cette conception a été réalisée de façon incrémentale et validée par des tests.

Réalisation et validation

Introduction

Ce chapitre présente la traduction effective de la conception en une application exécutable. Il ne décrit pas seulement les écrans obtenus : il relie les incréments du Product Backlog aux composants frontend, aux services métier, aux données persistées, aux processus d'infrastructure et aux contrôles qui permettent de vérifier leur comportement.

La réalisation a été menée par incréments dépendants. Chaque incrément stabilise une capacité démontrable avant d'ouvrir le suivant. Cette progression a d'abord sécurisé l'identité et l'isolation des organisations, puis le référentiel des stations, la communication OCPP, les opérations terrain, la recharge, le paiement et enfin le pilotage commercial. Elle aboutit à un premier parcours vertical :

**borne simulée → OCPP → disponibilité → alerte ou carte → session → tarif
→ paiement simulé → reçu.**

Les résultats présentés correspondent à l'état du dépôt au 28 juillet 2026. Une fonctionnalité est qualifiée de *réalisée* lorsqu'elle possède un contrat backend, une interface exploitable et des tests automatisés adaptés. Le terme *validée localement* ne signifie pas qu'elle est déjà certifiée pour la production : les limites relatives à une borne physique, à un prestataire de paiement réel, au déploiement et à la charge sont indiquées explicitement.

4.1 Environnement et organisation de la réalisation

4.1.1 Poste, outils et versions

Le développement est effectué sur un poste Windows avec PHP installé localement et Docker Desktop appuyé sur WSL2 pour les composants conteneurisés. Cette organisation

permet de conserver un cycle rapide pour React et Laravel tout en isolant la passerelle OCPP, le simulateur SAP et le simulateur de paiement. Git assure la traçabilité du code et GitHub centralise le dépôt distant.

Table 4.1 – Environnement technique effectivement utilisé

Domaine	Technologies	Usage dans le projet
Frontend	React 19, TypeScript 6, Vite 8	Application monopage, typage statique et compilation de production.
Interface	Ant Design 6, Recharts, Leaflet, GSAP	Composants, graphiques, cartes et animations ciblées.
État et échanges	TanStack Query, Axios, Zustand	Cache serveur, appels HTTP et états locaux transverses.
Backend	PHP 8.5, Laravel 13	API REST, logique métier, validation, autorisation et tâches planifiées.
Identité	Sanctum, Socialite, Spatie Permission	Session SPA, Google OAuth 2.0, rôles et permissions.
Données	PostgreSQL 16, Eloquent	Persistance transactionnelle, relations et migrations.
Temps réel	Reverb, Laravel Echo	Notifications et mises à jour poussées par WebSocket.
OCPP	Python 3.11, OCPP 1.6J, simulateur SAP	Passerelle WebSocket, messages de borne et scénarios de test.
Paieement	Contrat PaymentGateway, WireMock 3	Fournisseur externe simulé, webhooks et scénarios d'erreur.
Email	Resend, queue Laravel	Vérification d'email, invitations et notifications asynchrones.
Qualité	PHPUnit 12, Oxlint, TypeScript	Tests backend, analyse frontend et validation de compilation.
Documentation	L ^A T _E X, TikZ, OpenAPI prévu	Rapport, schémas et future spécification de l'API.

Le fichier `composer.json` exige PHP 8.3 ou supérieur, ce qui laisse la possibilité d'utiliser une version stable en déploiement même si le poste local exécute PHP 8.5. Les dépendances sont verrouillées par `composer.lock` et `pnpm-lock.yaml`. Les valeurs sensibles sont placées dans les fichiers `.env`, exclus de Git, tandis que les fichiers `.env.example` documentent uniquement les clés attendues.

4.1.2 Structure du dépôt

Le dépôt conserve une séparation physique entre les composants tout en proposant des commandes communes à sa racine :

frontend/	application React, pages, fonctionnalités, composants, styles et ressources visuelles ;
backend/	API Laravel, services métier, migrations, politiques, jobs et tests ;
ocpp-gateway/	passerelle Python entre les bornes et l'API interne Laravel ;
infra/ocpp/	composition Docker et profils du simulateur SAP ;
infra/payment-simulator/	serveur WireMock et contrats du faux fournisseur de paiement ;
scripts/	commandes d'orchestration, de provisioning et de pilotage des scénarios ;
docs/	rapport, diagrammes et futurs livrables d'exploitation.

L'audit statique du dépôt met en évidence 49 contrôleurs, 45 classes de validation HTTP, 24 ressources API, 39 modèles, 11 politiques et 47 migrations. Le frontend contient 39 pages React. Ces chiffres ne constituent pas une métrique de qualité en eux-mêmes ; ils montrent cependant que la logique a été répartie selon les responsabilités Laravel plutôt que concentrée dans quelques contrôleurs.

4.1.3 Processus de développement

Le fichier `package.json` racine sert de façade aux commandes des trois environnements. Une session de développement complète utilise les processus suivants :

Table 4.2 – Commandes principales de l’environnement local

Commande	Responsabilité
<code>pnpm dev:frontend</code>	Lance Vite sur le port 5173.
<code>pnpm dev:backend</code>	Lance l’API Laravel sur le port 8000.
<code>pnpm dev:queue</code>	Traite les emails et travaux asynchrones.
<code>pnpm dev:scheduler</code>	Exécute les règles périodiques.
<code>pnpm dev:reverb</code>	Lance le serveur WebSocket applicatif.
<code>pnpm ocpp:up</code>	Démarre la passerelle et le simulateur OCPP.
<code>pnpm ocpp:status</code>	Affiche l’état d’une borne simulée.
<code>pnpm ocpp:plug</code> / <code>ocpp:unplug</code>	Simule la connexion ou le retrait physique d’un câble.
<code>pnpm payment:up</code>	Démarre le fournisseur WireMock.
<code>pnpm build:frontend</code>	Type-check et compile le frontend.
<code>pnpm test:backend</code>	Exécute la campagne PHPUnit.

Le scheduler recalcule la disponibilité toutes les 30 secondes, expire les commandes OCPP toutes les 10 secondes, contrôle les SLA chaque minute et génère les maintenances récurrentes toutes les heures. Les abonnements clients et organisationnels sont également synchronisés toutes les heures. L’option `withoutOverlapping` empêche qu’une nouvelle occurrence commence alors que la précédente n’est pas terminée.

4.1.4 Gestion incrémentale avec Git

La chronologie des commits reflète les risques du produit. Les premières modifications ont initialisé l’authentification, PostgreSQL et l’isolation multi-tenant. Les stations, tarifs et sessions ont ensuite fourni les premiers objets métier. L’intégration OAuth, les invitations et les demandes de démonstration ont stabilisé l’entrée dans la plateforme. Les blocs OCPP, disponibilité et opérations ont été intégrés avant le paiement et les tableaux de bord. Enfin, le commissioning, la télémétrie vérifiée et les documents ont complété le parcours d’exploitation.

Chaque bloc fonctionnel est suivi d’un commit cohérent puis poussé sur la branche `main`. Cette discipline facilite l’identification d’une régression et conserve un historique

exploitable pour le rapport. Les fichiers d’environnement, les dépendances installées et les sorties de compilation ne sont pas versionnés.

4.1.5 État synthétique des incréments

Table 4.3 – État de réalisation au 28 juillet 2026

Inc.	Capacité démon- trable	Validation actuelle	Limite principale
INC-01	Comptes, Google OAuth, invitations, démo, onboarding et rôles	API et parcours web testés	Microsoft OAuth reporté.
INC-02	Stations, connecteurs, cartes, documents et commissioning	CRUD, politiques et création atomique testés	Pas de borne physique.
INC-03	OCPP, heartbeats, disponibilité, commandes et télémétrie	Simulateur SAP et projections testés	OCPP 2.0.1 hors périmètre.
INC-04	Alertes, interventions, maintenance, SLA et notifications	Cycles de vie et documents testés	Recette terrain à réaliser.
INC-05	Recharge, mesures, arrêt, paiement et reçu	Parcours vertical avec paiement simulé	Aucun débit bancaire réel.
INC-06	Tarifs, plans clients et contrat SaaS organisationnel	Règles, limites et factures testées	Souscription SaaS non encaissée.
INC-07	Dashboards, rapports, audit, paramètres et intégrations	APIs par rôle et exports testés	Finitions UX encore itératives.

Inc.	Capacité démon- trable	Validation actuelle	Limite principale
INC-08	Exports généralisés et préparation documentaire	CSV, JSON et PDF disponibles par périmètre	Firmware et CI/CD reportés.

4.2 Réalisation des fondations techniques

4.2.1 INC-01 : entrée, identité et multi-tenant

4.2.1.1 Parcours d'entrée

L'entrée dans ChargeTrackr dépend du type d'utilisateur. Un client peut créer son propre compte local, vérifier son adresse email, définir son mot de passe ou continuer avec Google. Un employé ne s'inscrit pas librement : l'administrateur de son organisation crée le compte opérateur ou technicien, puis une invitation temporaire permet au destinataire d'activer son accès et de choisir son mot de passe. Enfin, un futur administrateur commence par une demande de démonstration. Le super administrateur étudie la demande, crée l'organisation et émet l'invitation de l'administrateur.

Cette séparation résout deux problèmes métier. Premièrement, un compte employé ne peut pas s'attribuer lui-même une organisation ou un rôle privilégié. Deuxièmement, une demande de démonstration ne crée aucune ressource sensible avant la décision du super administrateur. Les invitations sont révocables, renouvelables et limitées dans le temps.

Le backend s'appuie sur Sanctum pour la session de la SPA et sur Socialite pour Google OAuth. Le service `GoogleAccountService` associe l'identité externe à un compte local sans importer les rôles Google. L'API locale reste propriétaire du rôle, des permissions, de l'organisation et de l'état du compte. Les formulaires d'inscription, de connexion, d'activation et de demande de démonstration possèdent des règles de limitation et des réponses d'erreur structurées.

4.2.1.2 Isolation organisationnelle

Les administrateurs, opérateurs et techniciens appartiennent à une seule organisation. Un client reste indépendant et peut utiliser les stations ou souscrire aux plans de

plusieurs organisations. Cette asymétrie est conservée dans le modèle de données : l'appartenance d'un employé détermine son périmètre, alors que les relations d'un client avec les organisations sont dérivées de ses abonnements et de ses sessions.

La protection ne repose pas sur le menu React. Le middleware `EnsureUserOrganizationScope`, les politiques et les requêtes Eloquent filtrent le périmètre côté serveur. Par exemple, un administrateur peut gérer les employés et consulter les clients de son organisation, mais il ne peut pas modifier un utilisateur d'une autre organisation en changeant simplement un identifiant dans l'URL. Les tests d'API couvrent ces refus d'accès.

4.2.1.3 Onboarding orienté vers la première valeur

Après sa première connexion, chaque rôle reçoit un parcours court qui confirme son contexte puis propose une première action utile. L'administrateur prépare l'identité de son organisation, invite un employé puis enregistre une station. L'opérateur rejoint directement la supervision, le technicien consulte son travail assigné et le client recherche une station. Le parcours peut être quitté et repris ; il ne bloque pas l'application.

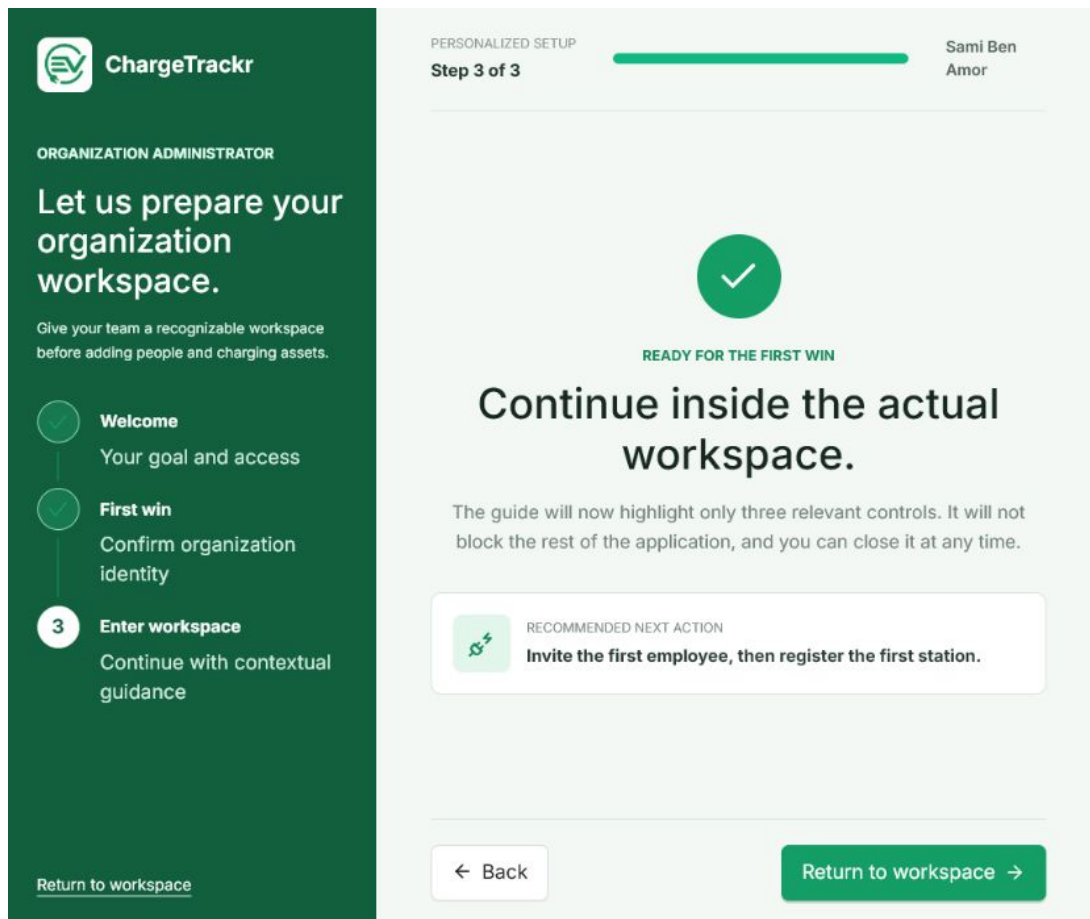


Figure 4.1 – Dernière étape de l'onboarding de l'administrateur

Les classes `AccountInvitationService` et `DemoProvisioningService`, les contrôleurs d'authentification et les politiques concrétisent l'incrément. Les campagnes `AuthApiTest`, `GoogleOAuthTest`, `DemoRequestWorkflowTest`, `OnboardingApiTest`, `OrganizationManagementApiTest` et `UserManagementApiTest` fournissent la preuve automatisée principale.

4.2.2 INC-02 : stations, connecteurs et cartographie

4.2.2.1 Référentiel et vues adaptées

Une station possède une référence opérationnelle unique, un nom, une adresse, une position, un constructeur, un modèle, une puissance, une version OCPP et un ensemble de connecteurs physiques. Un connecteur porte un identifiant OCPP unique dans sa station, un libellé lisible, un standard de prise, un type de courant et une puissance maximale.

Les écrans proposent des vues grille, liste, table et carte à partir du même contrat API. Les

filtres exploitent l'état calculé, le type de connecteur et la recherche textuelle. L'opérateur et l'administrateur peuvent gérer les actifs de leur organisation. Le technicien les consulte sans création, tandis que le client ne voit que les stations publiques pertinentes.

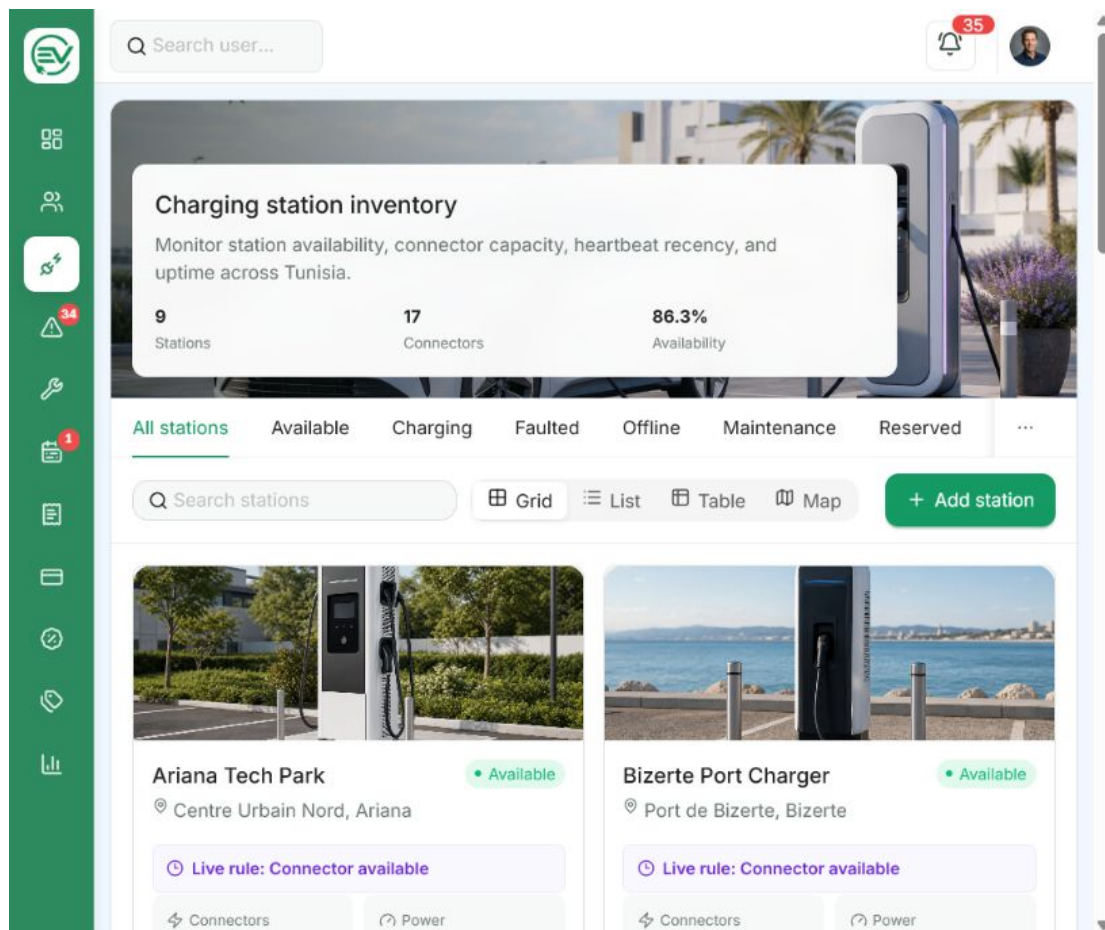


Figure 4.2 – Inventaire des stations avec indicateurs de disponibilité

Leaflet et OpenStreetMap permettent de placer un marqueur par clic ou glisser-déposer. Les coordonnées restent éditables pour traiter les cas où la géolocalisation est imprécise. Côté client, la préférence de rayon *Near me* limite la recherche autour de la position autorisée par le navigateur. Le frontend peut également copier les coordonnées ou ouvrir l'itinéraire dans un service cartographique externe.

4.2.2.2 Commissioning atomique

La simple création d'une ligne en base ne suffit pas à rendre une borne exploitable. Un assistant de commissioning regroupe l'identité de la station, son matériel, ses connecteurs et son profil de simulateur. La référence interne et l'identité OCPP sont contrôlées avant la transaction. Pour une cible locale, le backend prépare un profil que le simulateur SAP peut enregistrer.

Commission a station
Create the station, its physical connectors and its local OCPP simulator profile together.

Station | Hardware | Review

01 / STATION IDENTITY

Place the station on the network
Use a unique operational reference and set its exact public location.

Station name: Lac 1 Fast Hub

Internal reference: CT-TUN-101

Location name: Les Berges du Lac 1

City: Tunis

Street address: Rue du Lac Biwa, Tunis

Exact station position
Click the map or drag the marker. Coordinates remain editable when needed. [Enter manually](#)

Cancel Continue

Figure 4.3 – Première étape du commissioning guidé d’une station

`StationCommissioningService` exécute la création de la station, des connecteurs et des accès OCPP dans une transaction. Si une référence, une identité ou un connecteur est invalide, aucune création partielle ne subsiste. Le secret éventuel d’une borne physique n’est affiché qu’une fois et seul son condensat est conservé.

Les QR codes sont dérivés d’un jeton de connecteur destiné au parcours client. Le code affiché dans l’espace d’exploitation sert à imprimer l’étiquette physique ; le client utilise ensuite le scanner pour identifier le connecteur sans saisir manuellement sa référence. La plateforme ne considère toutefois pas le scan comme une preuve suffisante : la disponibilité et le démarrage restent validés par le backend et par OCPP.

Les documents de station sont persistés avec leur type, leur propriétaire et leur périmètre. Les autorisations distinguent contrats, garanties, notices et plans techniques. L’API ne retourne jamais un chemin arbitraire du système de fichiers ; le téléchargement passe par un contrôleur autorisé.

4.2.3 INC-03 : OCPP et disponibilité calculée

4.2.3.1 Connexion de la borne simulée

La passerelle Python reçoit les connexions WebSocket OCPP 1.6 JSON. Au démarrage, le simulateur envoie `BootNotification`, puis des `Heartbeat` périodiques et des `StatusNotification` pour chaque connecteur. La passerelle normalise le message et l'envoie à une API interne Laravel signée. Elle ne calcule pas l'état métier et n'écrit pas dans PostgreSQL.

`OcppEventIngestionService` enregistre l'événement brut utile à l'audit, met à jour les projections techniques et délègue les décisions à `AvailabilityProjectionService` ou `OcppTransactionProjectionService`. L'identifiant de la station présent dans l'URL WebSocket doit correspondre exactement à l'identité provisionnée.

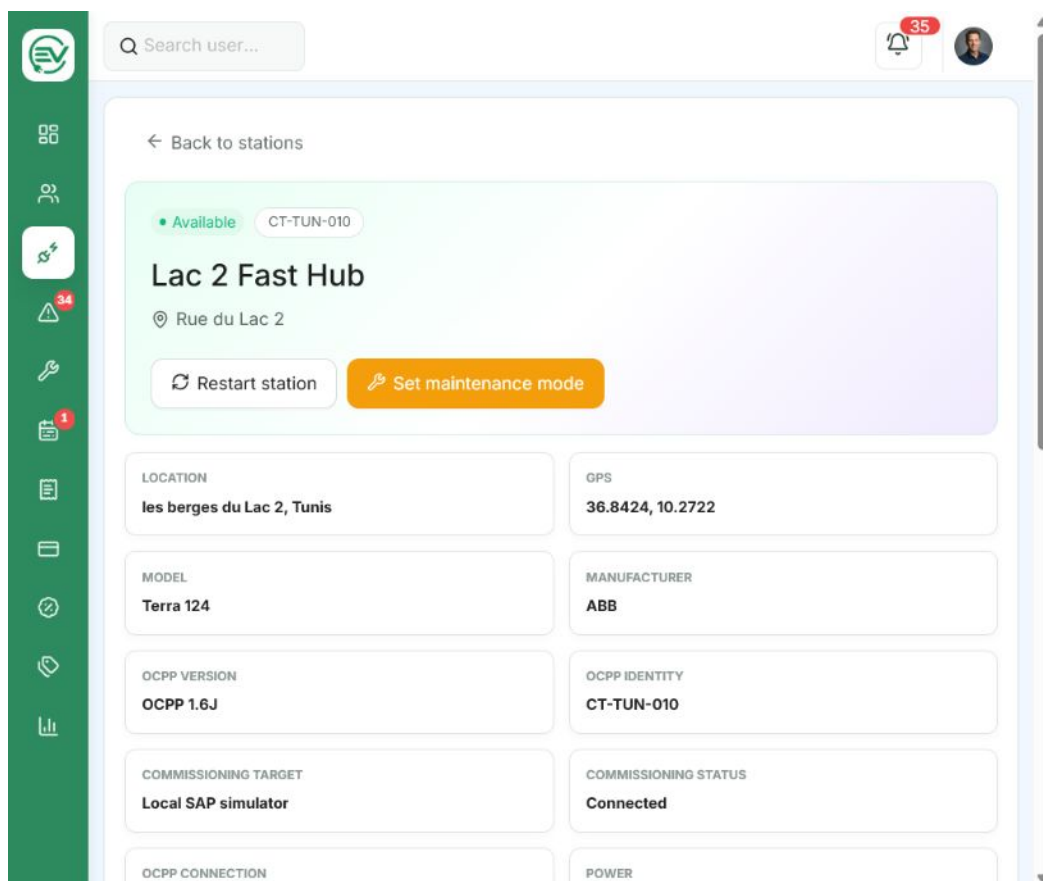


Figure 4.4 – Fiche d'une station reliée au simulateur SAP

4.2.3.2 Règles de projection

L'état affiché n'est pas un champ saisi manuellement. Il est reconstruit à partir des preuves disponibles selon un ordre de priorité. Une maintenance active ou une désactivation administrative peut rendre l'actif indisponible. Une absence de signal au-delà du délai configuré produit l'état déconnecté. Un code d'erreur OCPP actif produit l'état en défaut. Une transaction active rend le connecteur occupé. En l'absence de ces conditions, le dernier `StatusNotification` détermine notamment les états disponible, réservé ou indisponible.

Table 4.4 – Exemples de règles de disponibilité réalisées

État projeté	Preuve dominante	Conséquence
Disponible	Statut OCPP <code>Available</code> , signal récent et aucune contrainte prioritaire	Le connecteur peut être proposé au client.
Occupé	Transaction OCPP active ou statut <code>Charging</code>	Le connecteur n'est plus sélectionnable pour une nouvelle session.
Réservé	Statut <code>Reserved</code> ou réservation valide	L'accès est limité au bénéficiaire attendu.
Maintenance	Fenêtre de maintenance active ou commande de changement de disponibilité	Le connecteur est retiré du service.
En défaut	Code d'erreur ou statut <code>Faulted</code>	Une alerte opérationnelle est créée ou actualisée.
Déconnecté	Aucun heartbeat ou événement dans le délai configuré	La station est indisponible et une alerte de communication est ouverte.

Le recalcul est idempotent : recevoir plusieurs fois la même preuve ne crée pas une suite d'alertes identiques. Lorsque la communication revient, la projection est réévaluée et l'alerte peut être résolue selon sa règle. Les transitions sont horodatées afin de calculer l'uptime sur une période.

4.2.3.3 Commandes et télémetrie

Depuis la fiche station, un utilisateur autorisé peut demander **Reset**, **UnlockConnector** ou **ChangeAvailability**. Laravel crée d'abord une commande avec un état attendu, puis la passerelle la récupère et transmet l'appel à la borne. La réponse de la borne met à jour le résultat ; une tâche planifiée expire les commandes qui ne reçoivent pas de réponse. Le bouton n'affirme donc jamais qu'une action physique a réussi avant l'accusé OCPP.

MeterValues alimente la puissance et l'énergie réellement rapportées. L'interface distingue ces mesures des statistiques calculées à partir des sessions. Lorsqu'aucun **MeterValues** n'a été reçu, elle affiche *No signal* au lieu de générer une série visuelle artificielle.

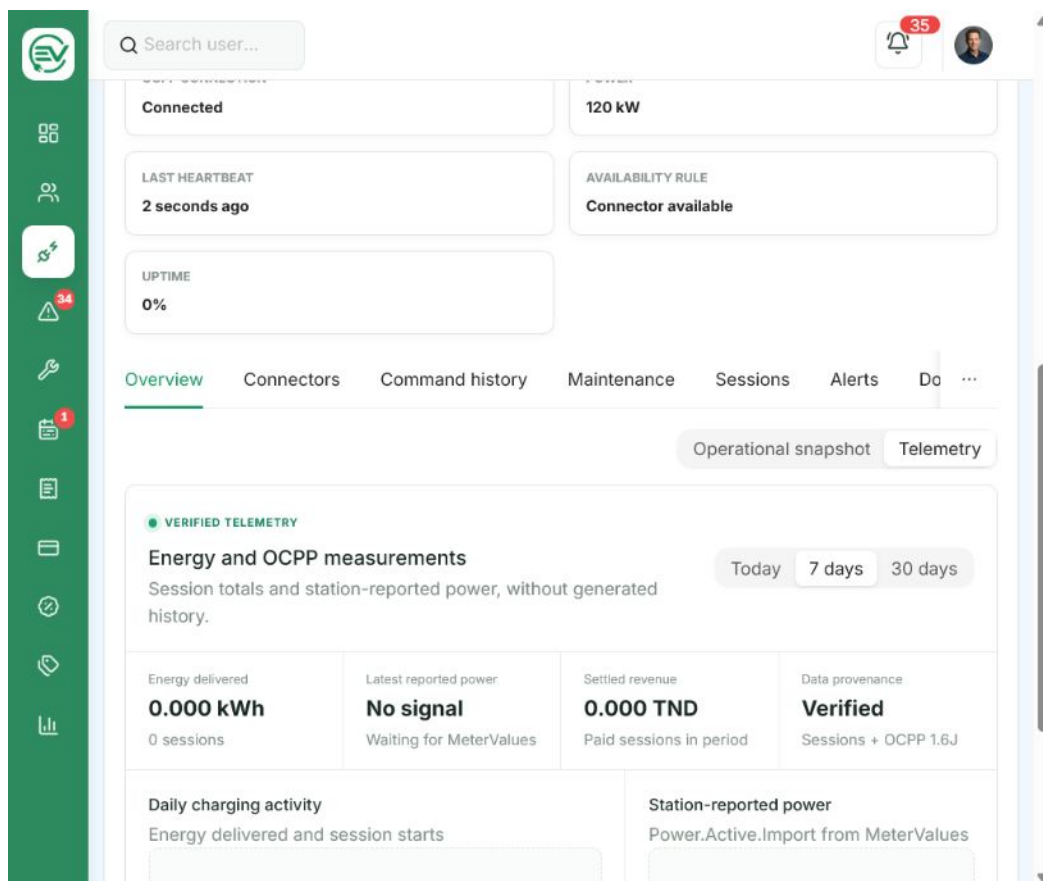


Figure 4.5 – Télémetrie vérifiée sans génération de mesures fictives

Les tests `OcppGatewayApiTest`, `OcppSimulatorFleetTest`, `OcppSupervisionApiTest`, `OcppTransactionProjectionTest`, `AvailabilityProjectionTest` et `StationTelemetryApiTest` vérifient les contrats, la déduplication, les transitions et l'isolation des organisations.

4.3 Réalisation des parcours métier

4.3.1 INC-04 : exploitation terrain

4.3.1.1 Alertes et qualification

Une alerte représente un symptôme opérationnel à traiter, et non une intervention. Elle possède une sévérité, un statut, une station, un contexte technique et une chronologie. Elle peut provenir automatiquement d'une perte de communication, d'un défaut OCPP ou d'une règle de maintenance, mais un utilisateur autorisé peut également créer une alerte manuelle.

L'interface utilise une disposition en deux panneaux : la liste filtrable reste visible pendant que le détail, les actions et l'historique de l'alerte sélectionnée sont consultés. Les filtres de sévérité et d'état ne modifient pas la donnée ; ils sont convertis en paramètres API. Les compteurs de la barre latérale sont dérivés des notifications non lues liées à ce contexte.

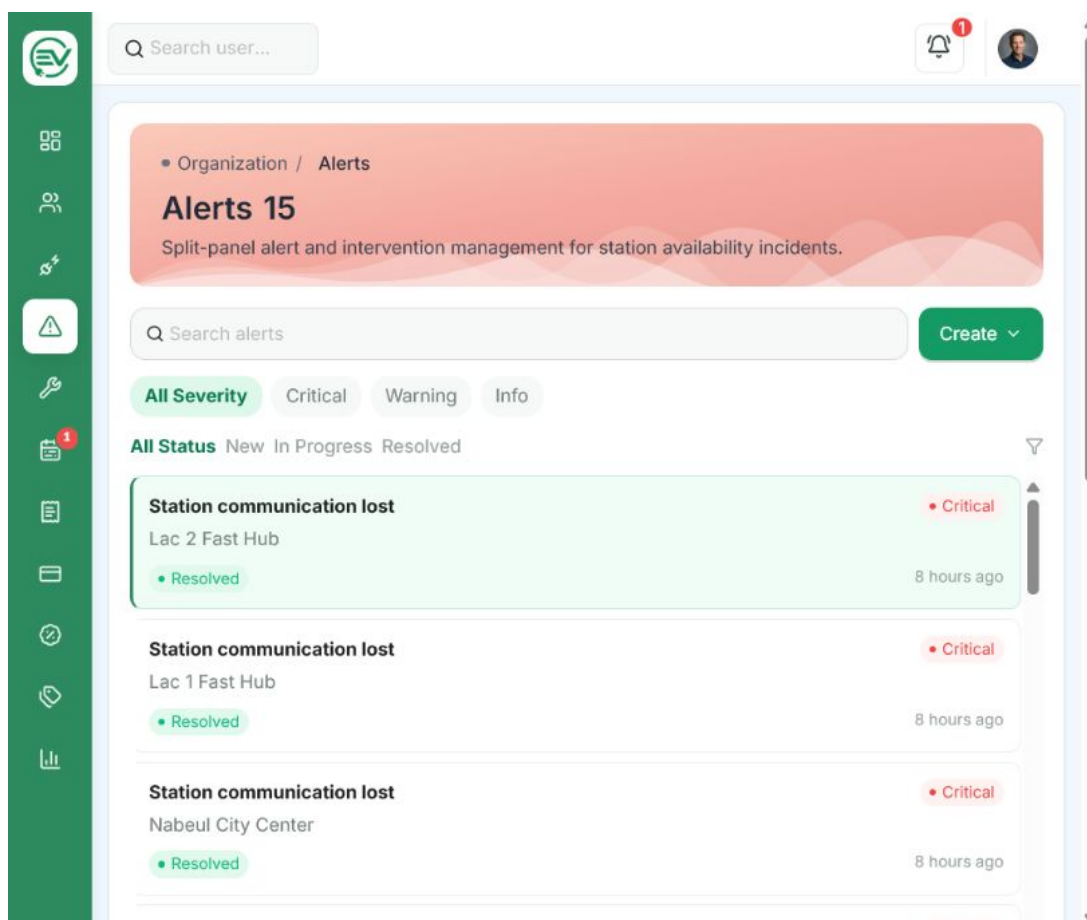


Figure 4.6 – Centre opérationnel des alertes

4.3.1.2 Interventions, maintenance et SLA

Une intervention matérialise le travail humain déclenché par une alerte ou une décision d'exploitation. L'opérateur la crée, choisit une priorité et l'assigne à un technicien de la même organisation. Le technicien accepte le travail, ajoute ses notes, documente le diagnostic, les actions et les pièces, joint des photos *avant/après*, puis soumet son rapport. L'opérateur contrôle ensuite le résultat et clôt l'intervention.

La maintenance répond à un besoin différent. Elle planifie une action préventive ou corrective sur une période, indépendamment de l'existence immédiate d'une alerte. `MaintenancePlanService` gère le plan, tandis que `MaintenanceLifecycleService` contrôle les transitions. Le job `GenerateMaintenanceOccurrences` matérialise les occurrences récurrentes.

`NotificationSlaService` analyse chaque minute les échéances approchantes ou dépassées. Les notifications sont enregistrées en base, diffusées par Reverb et éventuellement envoyées par email via la queue. L'utilisateur peut marquer une notification lue globalement ou dans le contexte de la page consultée.

Les documents d'alerte, d'intervention et de maintenance sont générés par le backend. Le rapport PDF reprend la référence, les acteurs, les dates, les preuves et l'historique sans dépendre d'une capture de l'écran. Les pièces jointes passent par des routes autorisées et sont supprimables selon le cycle de vie de la ressource.

4.3.2 INC-05 : recharge, transaction et paiement

4.3.2.1 Assistant de recharge client

Le client peut ouvrir l'assistant depuis *Start session*, depuis une fiche station ou depuis la carte. Dans les deux derniers cas, la station est déjà sélectionnée et le parcours commence directement au choix du connecteur. Le workflow se compose de cinq étapes :

1. sélectionner une station disponible ;
2. choisir un connecteur compatible et libre ;
3. brancher le câble et attendre la confirmation OCPP ;
4. autoriser le paiement simulé et vérifier le tarif applicable ;
5. envoyer la demande de démarrage distant et attendre la transaction OCPP.

Le passage de l'étape de branchement n'est pas manuel. Le frontend interroge la tentative de recharge et reçoit la mise à jour du connecteur ; il avance uniquement lorsque le

backend confirme une preuve OCPP compatible. Les pictogrammes de prises aident le client à reconnaître CCS2, Type 2, CHAdEMO ou un autre standard.

4.3.2.2 Cycle de session

`ChargingAttemptService` conserve le contexte entre les étapes et refuse une station, un connecteur ou un tarif devenus invalides. Après autorisation, `OcppCommandService` demande un `RemoteStartTransaction`. Le simulateur répond, puis `StartTransaction` devient la preuve de démarrage. Les `MeterValues` actualisent l'énergie, la puissance et l'estimation. Le client peut laisser la vue active en arrière-plan et y revenir depuis la page des sessions.

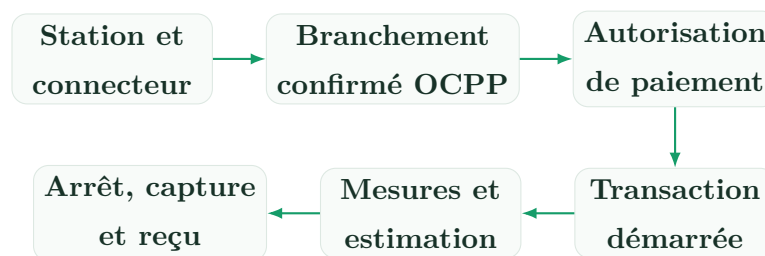


Figure 4.7 – Parcours vertical d'une recharge

L'arrêt peut être demandé par le client, par un opérateur autorisé, par la borne ou par une condition technique. `StopTransaction` clôt la transaction et fige les valeurs finales. Le coût n'est pas calculé dans React : `TariffResolver` sélectionne la règle applicable, puis le backend calcule les composantes énergie, temps, forfait et remise.

4.3.2.3 Adaptateur de paiement simulé

Le domaine dépend de l'interface `PaymentGateway`. Deux implémentations sont disponibles : `SimulatedPaymentAdapter`, utile aux tests unitaires, et `WireMockPaymentAdapter`, qui appelle un serveur HTTP externe local. Les opérations fonctionnelles sont l'autorisation, la capture, la libération et le débit direct.

`WireMock` reproduit des succès, refus, délais, erreurs serveur et webhooks. Le webhook est signé ; `PaymentWebhookSignature` vérifie son authenticité et `PaymentProviderEventService` assure l'idempotence. Un même événement rejoué ne capture donc pas deux fois une session. Le reçu est consultable dans l'application et exportable en PDF.

Ce choix ne simule pas une banque au niveau réglementaire. Il valide la frontière technique qui permettra d'ajouter ultérieurement `ClicToPay`, `e-DINAR/D17`, `GPGCheckout`,

Konnnect ou Flouci. L'adaptateur réel devra ajouter la conformité du prestataire, le rapprochement, la gestion des litiges et des remboursements.

4.3.3 INC-06 : tarification et commerce

4.3.3.1 Tarifs et plans de recharge

Un tarif appartient à une organisation et peut combiner un prix au kWh, un prix au temps, un coût fixe et des règles horaires. Les affectations définissent son périmètre : organisation, station ou connecteur. Lorsqu'une session commence, `TariffResolver` applique l'affectation la plus spécifique et conserve une référence au tarif retenu afin qu'une modification future ne change pas rétroactivement le calcul.

Les plans clients sont des avantages proposés par une organisation : remise sur la recharge, éventuel abonnement périodique et conditions d'accès. Un client peut cumuler des abonnements auprès de plusieurs réseaux, car son utilisation d'une borne ne le transforme pas automatiquement en client exclusif de son propriétaire. La simple recharge crée une relation transactionnelle ; l'abonnement demeure une décision explicite.

4.3.3.2 Abonnement SaaS de l'organisation

Le contrat commercial de l'organisation est volontairement séparé des paiements des conducteurs. Il détermine la période d'essai, le nombre maximal d'employés et de stations, les fonctions accessibles, la date de renouvellement et les factures adressées à l'organisation. `OrganizationEntitlementService` contrôle les limites avant la création d'un actif, tandis que `OrganizationSubscriptionLifecycleService` traite essai, activation, suspension, expiration et restauration.

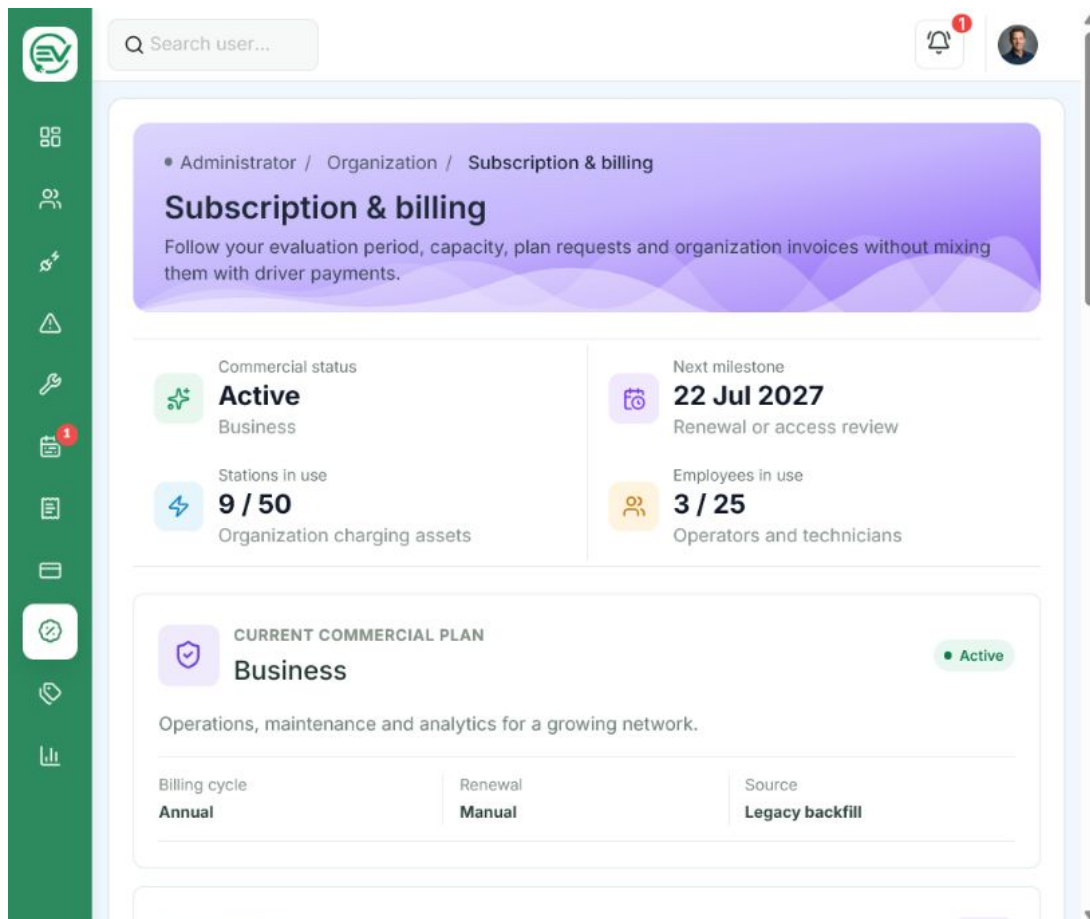


Figure 4.8 – Suivi de l'abonnement commercial d'une organisation

Le super administrateur gère le catalogue SaaS et le portefeuille global. L'administrateur consulte son offre, sa consommation de capacité et ses factures, puis émet une demande de changement. Cette demande évite qu'un administrateur modifie directement un contrat financier. L'encaissement SaaS reste simulé ou manuel dans le MVP.

4.3.4 INC-07 : pilotage, reporting et gouvernance

4.3.4.1 Tableaux de bord par rôle

DashboardService compose une réponse différente selon le rôle plutôt que de renvoyer le même tableau de bord avec des libellés modifiés. Le super administrateur supervise les organisations et les intégrations. L'administrateur observe les employés, clients, revenus, stations et performances de son organisation. L'opérateur suit disponibilité, incidents et sessions. Le technicien voit sa charge terrain et ses SLA. Le client consulte ses recharges, paiements et abonnements.

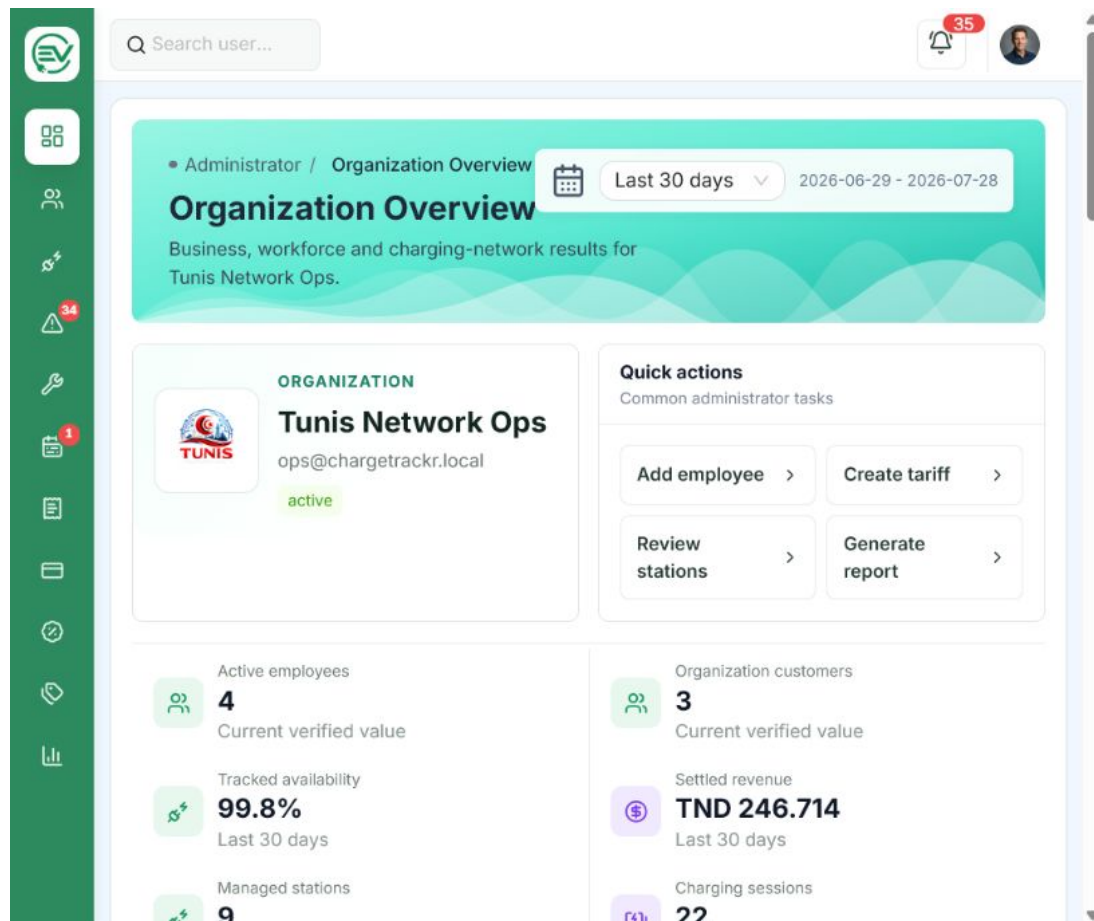


Figure 4.9 – Tableau de bord réel de l'administrateur

Les indicateurs utilisent les mêmes règles et périmètres que les écrans de détail. Par exemple, la disponibilité du tableau de bord passe par `AvailabilityMetrics` et ne moyenne pas des valeurs fictives du frontend. Les sélecteurs de période sont transformés en bornes temporelles côté API.

4.3.4.2 Rapports et échanges internes

Les espaces d'analyse sont adaptés au périmètre du rôle : `/reporting/platform`, `/organization`, `/operations` et `/field`. Les exports PDF, JSON et CSV réutilisent les mêmes filtres et autorisations. Les rapports internes sont des objets distincts des exports : un employé peut composer un rapport, joindre des fichiers, choisir un destinataire de son organisation, l'envoyer, puis suivre lecture et archivage.

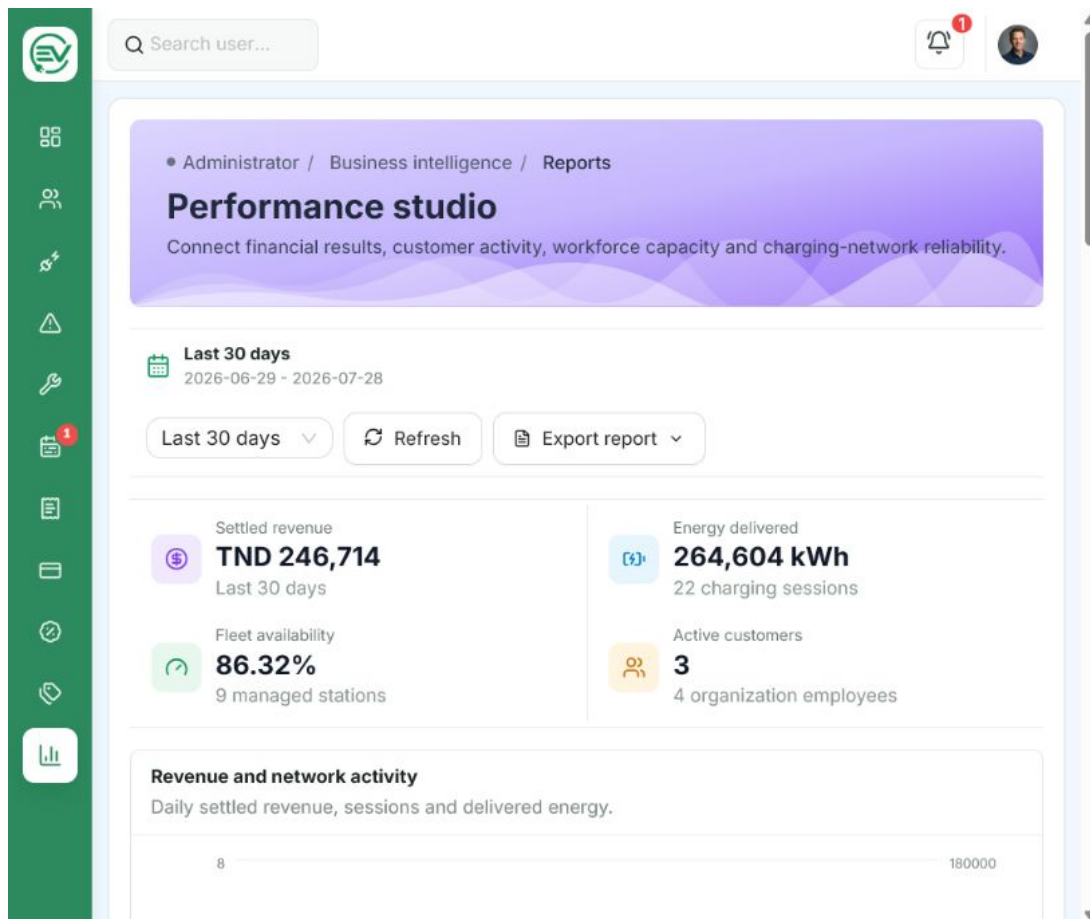


Figure 4.10 – Studio de performance de l’administrateur

4.3.4.3 Administration globale

Le super administrateur dispose d’un espace cohérent avec la navigation du produit, mais orienté opérations de plateforme : organisations, utilisateurs, rôles, permissions, demandes de démonstration, catalogue commercial, audit, intégrations et paramètres. Les secrets OAuth, OCPP, email ou paiement ne sont jamais renvoyés par les endpoints d’intégration. L’écran expose uniquement l’état de configuration et la date du dernier contrôle.

`PlatformSettingService` limite les paramètres modifiables à une liste validée. Les valeurs sensibles restent dans l’environnement. Toute modification globale est attribuée à son auteur dans le journal d’audit. Les paramètres personnels, tels que fuseau horaire, préférences de notification et rayon de recherche, restent séparés des contrôles globaux.

4.3.5 INC-08 : exports et évolutions différées

`ReportExportService` centralise les formats PDF, CSV et JSON. OpenSpout est disponible pour les formats tabulaires plus riches, tandis que DomPDF produit les documents visuels. Les reçus, factures et rapports opérationnels sont générés près de leur ressource afin que l'utilisateur les retrouve dans le workflow concerné, et non uniquement dans une page de rapports générique.

La gestion distante du firmware n'est pas simulée comme si elle était achevée. Elle nécessite une matrice de compatibilité matériel/logiciel, la signature des paquets, un hébergement sécurisé, le suivi de progression, une stratégie de retour arrière et une validation sur borne. Elle reste donc une évolution de l'INC-08, conformément au risque de 13 points associé à US-38.

4.4 Intégration et validation

4.4.1 Intégration bout en bout

L'intégration verticale est vérifiable sans accès direct du frontend à la borne ou au fournisseur de paiement. React appelle uniquement l'API publique Laravel. Laravel crée les intentions métier et les commandes. La passerelle OCPP échange avec le simulateur puis renvoie des événements signés. PostgreSQL conserve les preuves et projections. Reverb diffuse les changements utiles à l'interface. WireMock représente un système financier externe indépendant.

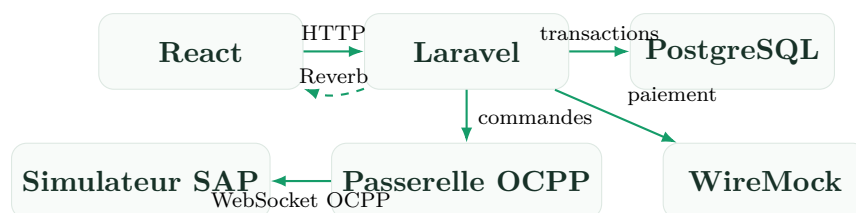


Figure 4.11 – Composants traversés par le MVP vertical

Cette structure a facilité le diagnostic. Un écran n'actualisant pas une donnée peut être analysé successivement au niveau de la requête React, de la réponse API, de la projection PostgreSQL, de l'événement reçu par la passerelle et du message émis par le simulateur. Le fournisseur de paiement peut de la même manière être observé sans modifier les tables depuis WireMock.

4.4.2 Stratégie de tests

La stratégie accorde la priorité aux invariants backend, car le frontend ne doit pas être la seule barrière contre une action interdite. Les tests utilisent une base SQLite en mémoire, une queue synchrone, un mailer en mémoire et un diffuseur nul. Ils restent ainsi reproductibles et n'envoient ni email réel ni événement WebSocket pendant la campagne.

Table 4.5 – Niveaux de validation appliqués

Niveau	Objet	Exemples	Preuve
Tests métier	Invariants, projections et transitions	disponibilité, transactions OCPP, tarifs, cycles commerciaux	Assertions PHPUnit.
Tests API	Validation, autorisation, sérialisation et effets	authentification, stations, alertes, rapports, paiement	Réponses et base vérifiées.
Tests d'isolation	Accès croisé entre organisations	utilisateurs, actifs, interventions, exports	Réponses 403/404 attendues.
Tests d'adaptateur	Contrat avec un service remplaçable	WireMock, webhook signé, Google OAuth simulé	Doubles et serveur local.
Tests OCPP	Lecture des profils et scénarios du simulateur	connexion, plug/unplug, transactions, projections	Node Test et PHPUnit.
Qualité frontend	Types, imports et règles statiques	compilation Vite et Oxlint	Commandes sans erreur.
Vérification visuelle	Mise en page et parcours représentatifs	dashboards, stations, cartes, formulaires, documents	Inspection navigateur/PDF.

4.4.3 Résultats mesurés

La campagne exécutée le 28 juillet 2026 produit les résultats suivants :

Table 4.6 – Résultats de validation du dépôt

Contrôle	Résultat	Portée
<code>php artisan test -compact</code>	201 tests, 1 489 assertions	37 classes de tests Feature et tests unitaires chargés par PHPUnit.
<code>pnpm test:ocpp-tools</code>	2 tests, 0 échec	Lecture et contrôle des états du simulateur.
<code>pnpm lint:frontend</code>	Succès	Analyse Oxlint du code TypeScript/React.
<code>pnpm build:frontend</code>	Succès	Type-check TypeScript et bundle Vite de production.
Inventaire des routes	186 routes, dont 177 API	75 GET, 67 POST, 23 PATCH/PUT et 12 DELETE.

La réussite de la compilation ne prouve pas seule la qualité UX, et le nombre de tests ne remplace pas une recette. Elle montre néanmoins que les contrats principaux, les migrations et la majorité des cycles de vie restent cohérents après l'intégration des incréments.

4.4.4 Anomalies représentatives et non-régression

Plusieurs anomalies rencontrées pendant la réalisation ont renforcé les contrôles :

- une requête depuis l'adresse locale du réseau pouvait recevoir une erreur 419 lorsque les domaines de session et les origines CSRF ne couvraient que `localhost`. La configuration a été rendue cohérente pour l'hôte local et l'adresse LAN de développement ;
- une double soumission involontaire d'une invitation déclenchait une erreur 429. Le bouton est verrouillé pendant la requête et les limites restent actives côté serveur ;
- un tri `created_at` combiné à un `GROUP BY` était accepté par certains moteurs mais refusé par PostgreSQL. La requête d'agrégation des photos d'intervention a été corrigée et couverte par un test ;
- `crypto.randomUUID()` échouait sur un navigateur mobile servi en HTTP non sécurisé. L'identifiant temporaire utilise désormais une solution compatible qui ne modifie pas l'identité métier du backend ;

- des URL WebSocket figées sur `localhost` empêchaient Reverb de fonctionner depuis un téléphone. Le frontend construit l'hôte de développement à partir de sa configuration plutôt que de supposer une machine unique.

Les réponses d'erreur de l'API sont normalisées pour ne pas exposer une trace SQL, un chemin local ou un secret. En environnement de production, `APP_DEBUG=false` reste obligatoire.

4.4.5 Performance et robustesse

Les listes volumineuses utilisent pagination et filtres côté serveur. Les relations nécessaires aux ressources API sont chargées explicitement afin de limiter les requêtes répétitives. Les agrégats de tableaux de bord sont calculés par période et périmètre plutôt que reconstruits dans le navigateur. Les travaux lents, notamment emails et certaines notifications, passent par la queue.

Pour OCPP, la passerelle conserve la connexion WebSocket mais délègue rapidement l'événement à Laravel. Le recalcul de disponibilité est idempotent et protégé contre le chevauchement. Les événements et mesures restent horodatés, ce qui permet d'appliquer un délai de non-réponse sans dépendre de l'heure du navigateur.

La compilation frontend signale toutefois plusieurs bundles supérieurs à 500 kio ; le principal fichier JavaScript atteint environ 767 kio. Ce résultat n'empêche pas le MVP local, mais justifie une optimisation avant production : découpage par route, chargement différé de PDF.js et du scanner QR, séparation des icônes et graphiques, puis mesure Lighthouse sur réseau mobile.

4.4.6 Sécurité appliquée

Les mesures suivantes sont effectivement présentes dans le code :

- session SPA protégée par Sanctum et cookie CSRF ;
- validation systématique des écritures par *Form Requests* ;
- politiques et scoping organisationnel côté serveur ;
- hachage des mots de passe et des secrets de station ;
- vérification des signatures de webhooks et déduplication des événements ;
- limitation des tentatives de connexion, d'activation et d'invitation ;
- routes internes OCPP séparées des routes publiques ;

- journal d’audit pour les actions sensibles ;
- fichiers et documents servis par contrôleurs autorisés ;
- secrets exclus du dépôt et masqués dans les réponses d’intégration.

Un audit de sécurité complet reste nécessaire avant exposition publique : rotation des clés, TLS/WSS, durcissement des cookies, en-têtes de sécurité, analyse des dépendances, tests IDOR systématiques, sauvegardes chiffrées et supervision des événements anormaux.

4.4.7 Limites de la validation

Frontière des résultats

- Le simulateur SAP valide les messages OCPP 1.6J, mais pas le comportement électrique, le modem ou le firmware d’une borne réelle.
- WireMock valide le contrat HTTP et les webhooks, mais aucune transaction bancaire ni exigence PCI DSS.
- Les tests automatisés n’ont pas encore été complétés par une campagne de charge OCPP multi-borne ni par des tests de performance formels.
- Le déploiement de production, TLS, sauvegardes, supervision et CI/CD ne sont pas encore qualifiés.
- La spécification OpenAPI/Swagger, les manuels et le procès-verbal de recette constituent des livrables distincts encore à finaliser.
- OCPP 2.0.1, Microsoft OAuth, le paiement réel et le firmware restent hors du MVP validé.

4.4.8 Traçabilité avec le backlog

Table 4.7 – Preuves de réalisation par incrément

Inc.	User stories	Preuves dans le code	Tests principaux
INC-01	US-01–04, 06, 27, 39	Auth, Google, invitations, démo, onboarding, polices	Auth, OAuth, onboarding, utilisateurs, organisations.

Inc.	User stories	Preuves dans le code	Tests principaux
INC-02	US-07–13, 37	Stations, connecteurs, cartes, commissioning, documents	Station API, commissioning, documents.
INC-03	US-14–16, 19	Gateway, ingestion, projection, commandes, télé-métrie	OCCP gateway, flotte, supervision, disponibilité, télé-métrie.
INC-04	US-20–23, 40	Alertes, interventions, maintenance, SLA, notifications	Alertes, rapports terrain, maintenance, notifications.
INC-05	US-24–26, 28–30	Tentatives, sessions, transactions, paiements, reçus	Recharge distante, session/paiement, webhook WireMock.
INC-06	US-31, 32, 41, 42	Tarifs, plans, abonnements clients et SaaS	Tarifs, plans, commercial organisationnel.
INC-07	US-05, 17, 18, 33–36	Dashboards, reporting, audit, paramètres, intégrations	Dashboard, rapports, gouvernance, recherche.
INC-08	US-34, 38	Exports PDF/J-SON/CSV ; firmware documenté mais différé	Exports couverts dans les tests de chaque domaine.

4.5 Documentation technique produite et restante

Le dépôt contient déjà les scripts d'installation, les configurations d'exemple, les commandes d'exploitation des simulateurs et le présent rapport. Les noms des routes et les ressources Laravel forment une base vérifiable pour produire une spécification OpenAPI 3.1. Celle-ci sera exposée avec Swagger UI pour l'exploration, mais le fichier OpenAPI versionné restera la source du livrable API.

Les livrables à finaliser sont le manuel utilisateur par rôle, le manuel administrateur, le cahier de tests, la procédure d'installation, la documentation des processus OCCP et le procès-verbal de recette. Ils devront reprendre les mêmes identifiants de user stories et

critères d'acceptation afin d'éviter une documentation déconnectée du produit.

Conclusion

La réalisation confirme la faisabilité de l'architecture proposée. Le dépôt contient une application React par rôle, une API Laravel protégée, un modèle PostgreSQL couvrant les domaines principaux, une passerelle OCPP indépendante et des adaptateurs pour les services externes. Le simulateur SAP produit des messages OCPP effectivement ingérés ; les services Laravel calculent ensuite les projections de disponibilité, les sessions, les alertes et les données de pilotage. Le paiement externe reste simulé, mais son contrat permet de remplacer WireMock sans réécrire le parcours de recharge.

La campagne locale atteint 201 tests backend et 1 489 assertions, complétés par deux tests des outils OCPP, le lint frontend et une compilation de production. Ces résultats valident le MVP vertical dans l'environnement du stage. Les travaux encore ouverts concernent principalement l'industrialisation, la mesure de charge, la documentation OpenAPI, la recette formelle, la connexion à un prestataire financier réel, l'essai sur une borne physique et la gestion sécurisée du firmware.

Captures des interfaces

5.1 Accueil, authentification et onboarding

Landing page ChargeTrackr

Présentation du produit et accès aux parcours publics

Figure 5.1 – Landing page ChargeTrackr

Connexion et création de compte

Connexion locale, Google OAuth et inscription client

Figure 5.2 – Connexion et création de compte

Onboarding personnalisé

Première connexion adaptée au rôle et à l'organisation

Figure 5.3 – Onboarding personnalisé

5.2 Tableaux de bord et supervision

Tableau de bord Super Administrateur

Pilotage global de la plateforme

Figure 5.4 – Tableau de bord Super Administrateur

Tableau de bord Administrateur

Indicateurs métier et réseau de l'organisation

Figure 5.5 – Tableau de bord Administrateur

Tableaux de bord Opérateur, Technicien et Client

Vues adaptées aux responsabilités de chaque rôle

Figure 5.6 – Tableaux de bord Opérateur, Technicien et Client

5.3 Bornes, connecteurs et cartographie

Inventaire des bornes

Vues carte, grille, liste et table

Figure 5.7 – Inventaire des bornes

Mise en service d'une borne

Création de la borne, des connecteurs et du profil OCPP

Figure 5.8 – Mise en service d'une borne

Détail et télémétrie d'une borne

État calculé, commandes, sessions, alertes et mesures OCPP

Figure 5.9 – Détail et télémétrie d'une borne

5.4 Recharge, sessions et paiement

Recherche d'une borne

Carte, filtres, distance et itinéraire

Figure 5.10 – Recherche d'une borne

Workflow de recharge

Sélection, branchement, autorisation, démarrage et suivi

Figure 5.11 – Workflow de recharge

Session active et paiement

Mesures en direct, arrêt, paiement simulé et reçu

Figure 5.12 – Session active et paiement

5.5 Alertes, interventions et maintenance

Gestion des alertes

Sévérité, SLA, assignation et historique

Figure 5.13 – Gestion des alertes

Intervention technique

Diagnostic, preuves, pièces, actions et rapport

Figure 5.14 – Intervention technique

Maintenance

Planification préventive et corrective

Figure 5.15 – Maintenance

5.6 Administration, gestion commerciale et rapports

Gestion des utilisateurs et organisations

Employés, clients, invitations et isolation multi-tenant

Figure 5.16 – Gestion des utilisateurs et organisations

Tarifs et abonnements

Tarification, plans clients et abonnement commercial

Figure 5.17 – Tarifs et abonnements

Rapports et analyses

Indicateurs par rôle, échange de rapports et exports

Figure 5.18 – Rapports et analyses

Profil et paramètres

Données personnelles, préférences et paramètres autorisés

Figure 5.19 – Profil et paramètres

5.7 Conclusion

Périmètre prévu

Résumer la cohérence visuelle, l'adaptation aux rôles et les parcours principaux illustrés dans ce chapitre.

Conclusion générale

Périmètre prévu

Présenter le bilan du stage, les objectifs atteints, les compétences mobilisées, les difficultés rencontrées et la valeur apportée par ChargeTrackr pour la supervision d'un réseau de bornes de recharge électrique.

Perspectives

Connexion à une borne physique, prestataire de paiement réel, OCPP 2.0.1, cartographie avancée, supervision du firmware, prédiction de pannes et assistant intelligent après stabilisation du cœur fonctionnel.

Bibliographie

- [1] TAC-TIC. Présentation de tac-tic. <https://tn.linkedin.com/company/tac-tic>. Page institutionnelle LinkedIn, consultée le 28 juillet 2026.
- [2] International Energy Agency. Global ev outlook 2026 – electric vehicle charging. <https://www.iea.org/reports/global-ev-outlook-2026/electric-vehicle-charging-chap-6-and-10>, 2026. Consulté le 28 juillet 2026.
- [3] Open Charge Alliance. Open charge point protocol 1.6. <https://openchargealliance.org/protocols/ocpp-protocols/ocpp-1-6/>. Consulté le 28 juillet 2026.
- [4] Ricardo Antunes et al. Open vehicle-to-everything management platform. *Information Systems*, 2024. https://ev4eu.eu/wp-content/uploads/2024/12/2024_OV2XMP_Information-Systems-journal_PPC.pdf, consulté le 28 juillet 2026.
- [5] Open Charge Alliance. Ocpp 1.6 compliancy testing tool – test case document. <https://openchargealliance.org/wp-content/uploads/2024/07/ComplianceTestTool-TestCaseDocument.pdf>, 2024. Consulté le 28 juillet 2026.
- [6] Open Charge Alliance. Improving uptime monitoring with ocpp. https://openchargealliance.org/wp-content/uploads/2024/08/improving_uptime_with_ocpp_v1_2.pdf, 2024. Livre blanc, consulté le 28 juillet 2026.
- [7] ChargePoint. Chargepoint software. <https://www.chargepoint.com/products/software>. Consulté le 28 juillet 2026.
- [8] Driivz. Ev charging operations management. <https://driivz.com/glossary/operations-management/>. Consulté le 28 juillet 2026.
- [9] Monta. Monta hub – charge point management system. <https://monta.com/en/products/hub-charge-point-management-system/>. Consulté le 28 juillet 2026.
- [10] LF Energy. Citrineos. <https://lfenergy.org/projects/citrineos/>. Consulté le 28 juillet 2026.
- [11] Open Charge Alliance. Ocpp 1.6 security whitepaper, 4th edition. <https://openchargealliance.org/ocpp-info-whitepapers/ocpp-1-6-security-whitepaper-4th-edition/>, 2026. Consulté le 28 juillet 2026.
- [12] SAP. E-mobility charging stations simulator. <https://github.com/SAP/e-mobility-charging-stations-simulator>. Dépôt GitHub officiel, consulté le 28 juillet 2026.

-
- [13] Leaflet. Leaflet – an open-source javascript library for interactive maps. <https://leafletjs.com/>. Consulté le 28 juillet 2026.
 - [14] OpenStreetMap contributors. Openstreetmap. <https://www.openstreetmap.org/>. Consulté le 28 juillet 2026.
 - [15] Ken Schwaber and Jeff Sutherland. The scrum guide. <https://scrumguides.org/scrum-guide.html>, 2020. Consulté le 28 juillet 2026.
 - [16] Meta Open Source. React documentation. <https://react.dev/>. Consulté en 2026.
 - [17] Laravel. Laravel documentation. <https://laravel.com/docs>. Consulté en 2026.
 - [18] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>. Consulté en 2026.